backend/src/index.ts:1-50

```
src/index.ts:0001 import 'dotenv/config';
src/index.ts:0002 import { createServer } from 'http';
src/index.ts:0003 import { closeDatabase } from './config/database';
src/index.ts:0004 import { getRuntimeDescriptor } from './config/platform-runtime';
src/index.ts:0005 import { bootstrapPersistence } from './bootstrap/persistence';
src/index.ts:0006 import { createControlPlaneApp } from './bootstrap/control-plane-app';
src/index.ts:0007 import { validateRuntimeEnvironment } from './bootstrap/runtime-env';
src/index.ts:0008 import logger from './utils/logger';
src/index.ts:0009 import { AgentGateway } from './websocket/agent-gateway';
src/index.ts:0010 import { DeviceGateway } from './websocket/device-gateway';
src/index.ts:0011 import { setAgentGateway } from './api/routes/agents.routes';
src/index.ts:0012 import { setDashboardAgentGateway } from './api/routes/dashboard.routes';
src/index.ts:0013 import {
src/index.ts:0014   initializeAgentSystem,
src/index.ts:0015   setAgentGateway as setAISystemAgentGateway
src/index.ts:0016 } from './api/routes/ai-system.routes';
src/index.ts:0017 import {
src/index.ts:0018   getDeviceById,
src/index.ts:0019   executeDeviceCommandInternal,
src/index.ts:0020   setDeviceCommandTransport
src/index.ts:0021 } from './api/routes/devices.routes';
src/index.ts:0022 import { scannerTaskService } from './services/scanner-task.service';
src/index.ts:0023 import { runtimeEmulatorService } from './services/runtime-emulator.service';
src/index.ts:0024 import { commandDispatcherService } from './services/command-dispatcher.service';
src/index.ts:0025 import { postgresShadowStoreService } from './services/postgres-shadow-store.service';
src/index.ts:0026 import { notificationDispatcherService } from './services/notification-dispatcher.service';
src/index.ts:0027 import { deviceControlService } from './services/device-control.service';
src/index.ts:0028 import { artifactStoreService } from './services/artifact-store.service';
src/index.ts:0029 import { jobQueueService } from './services/job-queue.service';
src/index.ts:0030 import { dashboardBroadcastService } from './services/dashboard-broadcast.service';
src/index.ts:0031
src/index.ts:0032 const runtime = getRuntimeDescriptor();
src/index.ts:0033 let agentGateway: AgentGateway | null = null;
src/index.ts:0034 let deviceGateway: DeviceGateway | null = null;
src/index.ts:0035
src/index.ts:0036 const app = createControlPlaneApp({
src/index.ts:0037   runtime,
src/index.ts:0038   getAgentCount: () => agentGateway?.getOnlineCount() || 0
src/index.ts:0039 });
src/index.ts:0040 const httpServer = createServer(app);
src/index.ts:0041
src/index.ts:0042 function configureRealtimeRuntime(): void {
src/index.ts:0043   if (runtime.gatewayEnabled) {
src/index.ts:0044     agentGateway = new AgentGateway(httpServer);
src/index.ts:0045     deviceGateway = new DeviceGateway(httpServer);
src/index.ts:0046     dashboardBroadcastService.setBroadcaster((event, data) => {
src/index.ts:0047       agentGateway?.broadcastToDashboards(event, data);
src/index.ts:0048     });
src/index.ts:0049     deviceGateway.setDashboardBroadcaster((event, data) => {
src/index.ts:0050       dashboardBroadcastService.broadcast(event, data);
```

backend/src/index.ts:51-100

```
src/index.ts:0051                        });
src/index.ts:0052                        deviceControlService.setGatewayEmitter((deviceId, command, params) =>
src/index.ts:0053                          deviceGateway!.sendCommand(deviceId, command, params)
src/index.ts:0054                        );
src/index.ts:0055
src/index.ts:0056                        setAgentGateway(agentGateway);
src/index.ts:0057                        setDashboardAgentGateway(agentGateway);
src/index.ts:0058                        setAISystemAgentGateway(agentGateway);
src/index.ts:0059                        commandDispatcherService.setAgentGateway(agentGateway);
src/index.ts:0060                        commandDispatcherService.setDeviceAdapter({
src/index.ts:0061                          getDevice: getDeviceById,
src/index.ts:0062                          executeCommand: executeDeviceCommandInternal
src/index.ts:0063                        });
src/index.ts:0064                        setDeviceCommandTransport((deviceId, command, params) =>
src/index.ts:0065                          deviceGateway!.sendCommand(deviceId, command, params)
src/index.ts:0066                        );
src/index.ts:0067                        scannerTaskService.setBroadcaster((event, data) => {
src/index.ts:0068                          dashboardBroadcastService.broadcast(event, data);
src/index.ts:0069                        });
src/index.ts:0070
src/index.ts:0071                        if (runtime.emulatorEnabled) {
src/index.ts:0072                          runtimeEmulatorService.start(agentGateway);
src/index.ts:0073                        } else {
src/index.ts:0074                          runtimeEmulatorService.stop();
src/index.ts:0075                        }
src/index.ts:0076                        return;
src/index.ts:0077                      }
src/index.ts:0078
src/index.ts:0079                      setAgentGateway(null);
src/index.ts:0080                      setDashboardAgentGateway(null);
src/index.ts:0081                      setAISystemAgentGateway(null as any);
src/index.ts:0082                      commandDispatcherService.setAgentGateway(null);
src/index.ts:0083                      commandDispatcherService.setDeviceAdapter(null);
src/index.ts:0084                      setDeviceCommandTransport(null);
src/index.ts:0085                      deviceControlService.setGatewayEmitter(null);
src/index.ts:0086                      dashboardBroadcastService.setBroadcaster(null);
src/index.ts:0087                      scannerTaskService.setBroadcaster(() => undefined);
src/index.ts:0088                      runtimeEmulatorService.stop();
src/index.ts:0089                    }
src/index.ts:0090
src/index.ts:0091                    async function start(): Promise<void> {
src/index.ts:0092                      const port = Number(process.env.PORT || 8080);
src/index.ts:0093
src/index.ts:0094                      try {
src/index.ts:0095                        const envReport = validateRuntimeEnvironment();
src/index.ts:0096                        await bootstrapPersistence();
src/index.ts:0097                        configureRealtimeRuntime();
src/index.ts:0098
src/index.ts:0099                        if (runtime.workerEnabled) {
src/index.ts:0100                          await scannerTaskService.startWorker();
```

backend/src/index.ts:101-150

```
src/index.ts:0101                          }
src/index.ts:0102
src/index.ts:0103                          notificationDispatcherService.startWorker();
src/index.ts:0104
src/index.ts:0105                          if (runtime.schedulerEnabled) {
src/index.ts:0106                            postgresShadowStoreService.startResyncLoop();
src/index.ts:0107                          }
src/index.ts:0108
src/index.ts:0109                          if (runtime.apiEnabled) {
src/index.ts:0110                            try {
src/index.ts:0111                              await initializeAgentSystem();
src/index.ts:0112                              logger.info('Multi-Agent Decision System initialized');
src/index.ts:0113                            } catch (agentError: any) {
src/index.ts:0114                              logger.warn('Multi-Agent System initialization failed, continuing without AI agents', {
src/index.ts:0115                                error: agentError.message
src/index.ts:0116                              });
src/index.ts:0117                            }
src/index.ts:0118                          }
src/index.ts:0119
src/index.ts:0120                          httpServer.listen(port, () => {
src/index.ts:0121                            logger.info('AetherGuard Control Platform started', {
src/index.ts:0122                              port,
src/index.ts:0123                              runtime,
src/index.ts:0124                              queueBackend: jobQueueService.backend(),
src/index.ts:0125                              artifactStore: artifactStoreService.describe(),
src/index.ts:0126                              shadowStore: postgresShadowStoreService.describe(),
src/index.ts:0127                              envWarnings: envReport.warnings
src/index.ts:0128                            });
src/index.ts:0129                          });
src/index.ts:0130                        } catch (error: any) {
src/index.ts:0131                          logger.error('Failed to start server', { error: error?.message || error });
src/index.ts:0132                          process.exit(1);
src/index.ts:0133                        }
src/index.ts:0134                      }
src/index.ts:0135
src/index.ts:0136                      async function shutdown(): Promise<void> {
src/index.ts:0137                        logger.info('Shutting down...');
src/index.ts:0138
src/index.ts:0139                        try {
src/index.ts:0140                          scannerTaskService.stopWorker();
src/index.ts:0141                          notificationDispatcherService.stopWorker();
src/index.ts:0142
src/index.ts:0143                          if (agentGateway) {
src/index.ts:0144                            agentGateway.close();
src/index.ts:0145                            agentGateway = null;
src/index.ts:0146                          }
src/index.ts:0147                          if (deviceGateway) {
src/index.ts:0148                            deviceGateway.close();
src/index.ts:0149                            deviceGateway = null;
src/index.ts:0150                          }
```

backend/src/index.ts:151-162 | backend/src/websocket/agent-gateway.ts:1-38

```
src/index.ts:0151                          dashboardBroadcastService.setBroadcaster(null);
src/index.ts:0152
src/index.ts:0153                          runtimeEmulatorService.stop();
src/index.ts:0154
src/index.ts:0155                          await new Promise<void>((resolve, reject) => {
src/index.ts:0156                            httpServer.close((error) => {
src/index.ts:0157                              if (error) {
src/index.ts:0158                                reject(error);
src/index.ts:0159                                return;
src/index.ts:0160                              }
src/index.ts:0161                              resolve();
src/index.ts:0162                            });
src/websocket/agent-gateway.ts:0001      /**
src/websocket/agent-gateway.ts:0002       * WebSocket Agent Gateway - SECURE VERSION
src/websocket/agent-gateway.ts:0003       *
src/websocket/agent-gateway.ts:0004       * SECURITY FIXES:
src/websocket/agent-gateway.ts:0005       * - Server-side agent authentication
src/websocket/agent-gateway.ts:0006       * - Command signing with HMAC
src/websocket/agent-gateway.ts:0007       * - Comprehensive audit logging
src/websocket/agent-gateway.ts:0008       * - Rate limiting (Redis-backed with memory fallback)
src/websocket/agent-gateway.ts:0009       * - Strict IP validation
src/websocket/agent-gateway.ts:0010       * - Redis degradation support
src/websocket/agent-gateway.ts:0011       */
src/websocket/agent-gateway.ts:0012
src/websocket/agent-gateway.ts:0013      import { Server as SocketIOServer, Socket } from 'socket.io';
src/websocket/agent-gateway.ts:0014      import { Server as HttpServer } from 'http';
src/websocket/agent-gateway.ts:0015      import { isIPv4, isIPv6 } from 'net';
src/websocket/agent-gateway.ts:0016      import logger from '../utils/logger';
src/websocket/agent-gateway.ts:0017      import { AgentStatus, ThreatAlert } from '../agent';
src/websocket/agent-gateway.ts:0018      import type { AgentHeartbeat as ProbeAgentHeartbeat } from '../ai-agents';
src/websocket/agent-gateway.ts:0019      import { verifyTokenAsync } from '../middleware/auth.middleware';
src/websocket/agent-gateway.ts:0020      import type { AuthenticatedUser } from '../middleware/auth.middleware';
src/websocket/agent-gateway.ts:0021      import agentAuthService from '../security/agent-auth.service';
src/websocket/agent-gateway.ts:0022      import type { AuthenticatedAgent } from '../security/agent-auth.service';
src/websocket/agent-gateway.ts:0023      import commandSigningService from '../security/command-signing.service';
src/websocket/agent-gateway.ts:0024      import auditLogger from '../security/audit-logger.service';
src/websocket/agent-gateway.ts:0025      import { getTrafficStats, getAgentSystem } from '../api/routes/ai-system.routes';
src/websocket/agent-gateway.ts:0026      import {
src/websocket/agent-gateway.ts:0027        isTestRuntime,
src/websocket/agent-gateway.ts:0028        wsTestAgentAdapter,
src/websocket/agent-gateway.ts:0029        wsTestDashboardUserAdapter
src/websocket/agent-gateway.ts:0030      } from '../testing/test-runtime-adapter';
src/websocket/agent-gateway.ts:0031      import { commandDispatcherService } from '../services/command-dispatcher.service';
src/websocket/agent-gateway.ts:0032      import { socketRateLimitService } from '../services/socket-rate-limit.service';
src/websocket/agent-gateway.ts:0033      import { detectionPipelineService } from '../services/detection-pipeline.service';
src/websocket/agent-gateway.ts:0034      import { edgeAutoService } from '../services/edge-auto.service';
src/websocket/agent-gateway.ts:0035      import type { TenantContext } from '../core/tenant/tenant-context';
src/websocket/agent-gateway.ts:0036      import {
src/websocket/agent-gateway.ts:0037        buildPlatformTenantContext,
src/websocket/agent-gateway.ts:0038        resolveAgentTenantContext,
```

backend/src/websocket/agent-gateway.ts:39-88

```
src/websocket/agent-gateway.ts:0039       resolveUserTenantContext,
src/websocket/agent-gateway.ts:0040       PLATFORM_TENANT_ID
src/websocket/agent-gateway.ts:0041     } from '../core/tenant/tenant-context';
src/websocket/agent-gateway.ts:0042     import type {
src/websocket/agent-gateway.ts:0043       AgentClientToServerEvents,
src/websocket/agent-gateway.ts:0044       AgentRegisterPayload,
src/websocket/agent-gateway.ts:0045       AgentServerToClientEvents
src/websocket/agent-gateway.ts:0046     } from './socket-events';
src/websocket/agent-gateway.ts:0047     import {
src/websocket/agent-gateway.ts:0048       validateAgentHeartbeatPayload,
src/websocket/agent-gateway.ts:0049       validateAgentProtocolObservationPayload,
src/websocket/agent-gateway.ts:0050       validateAgentRegisterPayload,
src/websocket/agent-gateway.ts:0051       validateDashboardAgentCommandPayload,
src/websocket/agent-gateway.ts:0052       validateDashboardIpCommandPayload
src/websocket/agent-gateway.ts:0053     } from './socket-payload-validator';
src/websocket/agent-gateway.ts:0054     import type { JsonObject } from '../types/json';
src/websocket/agent-gateway.ts:0055     import type { ProtocolEvaluateResponse } from '../types/edge-auto';
src/websocket/agent-gateway.ts:0056
src/websocket/agent-gateway.ts:0057     // Rate limiting configuration
src/websocket/agent-gateway.ts:0058     const RATE LIMIT WINDOW = 60000; // 1 minute
src/websocket/agent-gateway.ts:0059     const MAX REQUESTS PER WINDOW = 100;
src/websocket/agent-gateway.ts:0060     const USER_RATE_LIMIT_WINDOW = 60000; // 1 minute
src/websocket/agent-gateway.ts:0061     const MAX_USER_REQUESTS_PER_WINDOW = 150; // Slightly higher than per-socket limit
src/websocket/agent-gateway.ts:0062
src/websocket/agent-gateway.ts:0063     type GatewayClientType = 'agent' | 'dashboard';
src/websocket/agent-gateway.ts:0064     type GatewayAuthenticatedAgent = AuthenticatedAgent & Pick<
src/websocket/agent-gateway.ts:0065       AgentRegisterPayload,
src/websocket/agent-gateway.ts:0066       'architecture' | 'arch' | 'nodeKind' | 'deploymentClass'
src/websocket/agent-gateway.ts:0067     >;
src/websocket/agent-gateway.ts:0068     type AgentCommandPayload = JsonObject;
src/websocket/agent-gateway.ts:0069     type AgentCommandMetadata = JsonObject;
src/websocket/agent-gateway.ts:0070     type DashboardEventRecord = Record<string, unknown>;
src/websocket/agent-gateway.ts:0071     type DashboardEventPayload = DashboardEventRecord | DashboardEventRecord[];
src/websocket/agent-gateway.ts:0072     type AgentRuntimeMode = ConnectedAgent['runtimeMode'];
src/websocket/agent-gateway.ts:0073
src/websocket/agent-gateway.ts:0074     interface SyntheticAgentCommand {
src/websocket/agent-gateway.ts:0075       type: string;
src/websocket/agent-gateway.ts:0076       payload?: AgentCommandPayload;
src/websocket/agent-gateway.ts:0077       userId?: string;
src/websocket/agent-gateway.ts:0078       metadata?: AgentCommandMetadata;
src/websocket/agent-gateway.ts:0079     }
src/websocket/agent-gateway.ts:0080
src/websocket/agent-gateway.ts:0081     interface GatewaySocketData {
src/websocket/agent-gateway.ts:0082       clientType?: GatewayClientType;
src/websocket/agent-gateway.ts:0083       agent?: GatewayAuthenticatedAgent;
src/websocket/agent-gateway.ts:0084       user?: AuthenticatedUser;
src/websocket/agent-gateway.ts:0085       tenantContext?: TenantContext;
src/websocket/agent-gateway.ts:0086     }
src/websocket/agent-gateway.ts:0087
src/websocket/agent-gateway.ts:0088     type GatewaySocket = Socket<
```

backend/src/websocket/agent-gateway.ts:89-138

```
src/websocket/agent-gateway.ts:0089       AgentClientToServerEvents,
src/websocket/agent-gateway.ts:0090       AgentServerToClientEvents,
src/websocket/agent-gateway.ts:0091       Record<string, never>,
src/websocket/agent-gateway.ts:0092       GatewaySocketData
src/websocket/agent-gateway.ts:0093     >;
src/websocket/agent-gateway.ts:0094
src/websocket/agent-gateway.ts:0095     interface AgentListItem {
src/websocket/agent-gateway.ts:0096       agentId: string;
src/websocket/agent-gateway.ts:0097       hostname: string;
src/websocket/agent-gateway.ts:0098       platform: string;
src/websocket/agent-gateway.ts:0099       architecture?: string;
src/websocket/agent-gateway.ts:0100       nodeKind?: 'cloud' | 'edge' | 'vehicle';
src/websocket/agent-gateway.ts:0101       deploymentClass?: 'standard' | 'edge' | 'vehicle';
src/websocket/agent-gateway.ts:0102       runtimeMode?: AgentRuntimeMode;
src/websocket/agent-gateway.ts:0103       tenantId: string;
src/websocket/agent-gateway.ts:0104       registration?: DashboardEventRecord;
src/websocket/agent-gateway.ts:0105       lastHeartbeat: string;
src/websocket/agent-gateway.ts:0106       status: AgentStatus | null;
src/websocket/agent-gateway.ts:0107       isOnline: boolean;
src/websocket/agent-gateway.ts:0108       synthetic: boolean;
src/websocket/agent-gateway.ts:0109     }
src/websocket/agent-gateway.ts:0110
src/websocket/agent-gateway.ts:0111     interface TrafficProbeStats {
src/websocket/agent-gateway.ts:0112       packetsProcessed: number;
src/websocket/agent-gateway.ts:0113       packetsAllowed: number;
src/websocket/agent-gateway.ts:0114       packetsBlocked: number;
src/websocket/agent-gateway.ts:0115       packetsInspected: number;
src/websocket/agent-gateway.ts:0116       bytesProcessed: number;
src/websocket/agent-gateway.ts:0117       activeFlows: JsonObject[];
src/websocket/agent-gateway.ts:0118     }
src/websocket/agent-gateway.ts:0119
src/websocket/agent-gateway.ts:0120     function asRecord(input: unknown): Record<string, unknown> | null {
src/websocket/agent-gateway.ts:0121       return input && typeof input === 'object' && !Array.isArray(input)
src/websocket/agent-gateway.ts:0122         ? input as Record<string, unknown>
src/websocket/agent-gateway.ts:0123         : null;
src/websocket/agent-gateway.ts:0124     }
src/websocket/agent-gateway.ts:0125
src/websocket/agent-gateway.ts:0126     function asJsonObject(input: unknown): JsonObject | null {
src/websocket/agent-gateway.ts:0127       return input && typeof input === 'object' && !Array.isArray(input)
src/websocket/agent-gateway.ts:0128         ? input as JsonObject
src/websocket/agent-gateway.ts:0129         : null;
src/websocket/agent-gateway.ts:0130     }
src/websocket/agent-gateway.ts:0131
src/websocket/agent-gateway.ts:0132     function optionalString(value: unknown): string | undefined {
src/websocket/agent-gateway.ts:0133       return typeof value === 'string' && value.trim() ? value : undefined;
src/websocket/agent-gateway.ts:0134     }
src/websocket/agent-gateway.ts:0135
src/websocket/agent-gateway.ts:0136     function optionalNumber(value: unknown): number | undefined {
src/websocket/agent-gateway.ts:0137       const numeric = Number(value);
src/websocket/agent-gateway.ts:0138       return Number.isFinite(numeric) ? numeric : undefined;
```

backend/src/websocket/agent-gateway.ts:139-188

```
src/websocket/agent-gateway.ts:0139     }
src/websocket/agent-gateway.ts:0140
src/websocket/agent-gateway.ts:0141     function optionalStringArray(value: unknown): string[] | undefined {
src/websocket/agent-gateway.ts:0142       if (!Array.isArray(value)) {
src/websocket/agent-gateway.ts:0143         return undefined;
src/websocket/agent-gateway.ts:0144       }
src/websocket/agent-gateway.ts:0145
src/websocket/agent-gateway.ts:0146       const normalized = value.filter((item): item is string => typeof item === 'string' && item.trim().length > 0);
src/websocket/agent-gateway.ts:0147       return normalized.length > 0 ? normalized : undefined;
src/websocket/agent-gateway.ts:0148     }
src/websocket/agent-gateway.ts:0149
src/websocket/agent-gateway.ts:0150     function asJsonObjectArray(value: unknown): JsonObject[] {
src/websocket/agent-gateway.ts:0151       if (!Array.isArray(value)) {
src/websocket/agent-gateway.ts:0152         return [];
src/websocket/agent-gateway.ts:0153       }
src/websocket/agent-gateway.ts:0154
src/websocket/agent-gateway.ts:0155       return value
src/websocket/agent-gateway.ts:0156         .map((item) => asJsonObject(item))
src/websocket/agent-gateway.ts:0157         .filter((item): item is JsonObject => item !== null);
src/websocket/agent-gateway.ts:0158     }
src/websocket/agent-gateway.ts:0159
src/websocket/agent-gateway.ts:0160     function toJsonObject(input: Record<string, unknown>): JsonObject {
src/websocket/agent-gateway.ts:0161       return JSON.parse(JSON.stringify(input)) as JsonObject;
src/websocket/agent-gateway.ts:0162     }
src/websocket/agent-gateway.ts:0163
src/websocket/agent-gateway.ts:0164     function getErrorMessage(error: unknown): string {
src/websocket/agent-gateway.ts:0165       if (error instanceof Error) {
src/websocket/agent-gateway.ts:0166         return error.message;
src/websocket/agent-gateway.ts:0167       }
src/websocket/agent-gateway.ts:0168
src/websocket/agent-gateway.ts:0169       const record = asRecord(error);
src/websocket/agent-gateway.ts:0170       if (record && typeof record.message === 'string') {
src/websocket/agent-gateway.ts:0171         return record.message;
src/websocket/agent-gateway.ts:0172       }
src/websocket/agent-gateway.ts:0173
src/websocket/agent-gateway.ts:0174       return String(error ?? 'unknown_error');
src/websocket/agent-gateway.ts:0175     }
src/websocket/agent-gateway.ts:0176
src/websocket/agent-gateway.ts:0177     function normalizeTrafficStats(input: JsonObject | undefined): TrafficProbeStats | null {
src/websocket/agent-gateway.ts:0178       if (!input) {
src/websocket/agent-gateway.ts:0179         return null;
src/websocket/agent-gateway.ts:0180       }
src/websocket/agent-gateway.ts:0181
src/websocket/agent-gateway.ts:0182       return {
src/websocket/agent-gateway.ts:0183         packetsProcessed: optionalNumber(input.packetsProcessed) || 0,
src/websocket/agent-gateway.ts:0184         packetsAllowed: optionalNumber(input.packetsAllowed) || 0,
src/websocket/agent-gateway.ts:0185         packetsBlocked: optionalNumber(input.packetsBlocked) || 0,
src/websocket/agent-gateway.ts:0186         packetsInspected: optionalNumber(input.packetsInspected) || 0,
src/websocket/agent-gateway.ts:0187         bytesProcessed: optionalNumber(input.bytesProcessed) || 0,
src/websocket/agent-gateway.ts:0188         activeFlows: asJsonObjectArray(input.activeFlows)
```

backend/src/websocket/agent-gateway.ts:189-238

```
src/websocket/agent-gateway.ts:0189       };
src/websocket/agent-gateway.ts:0190     }
src/websocket/agent-gateway.ts:0191
src/websocket/agent-gateway.ts:0192     function getHandshakeAuth(socket: GatewaySocket): Record<string, unknown> {
src/websocket/agent-gateway.ts:0193       return asRecord(socket.handshake.auth) || {};
src/websocket/agent-gateway.ts:0194     }
src/websocket/agent-gateway.ts:0195
src/websocket/agent-gateway.ts:0196     function getCookieHeader(cookieHeader: string | string[] | undefined): string | undefined {
src/websocket/agent-gateway.ts:0197       if (Array.isArray(cookieHeader)) {
src/websocket/agent-gateway.ts:0198         return cookieHeader.join('; ');
src/websocket/agent-gateway.ts:0199       }
src/websocket/agent-gateway.ts:0200
src/websocket/agent-gateway.ts:0201       return cookieHeader;
src/websocket/agent-gateway.ts:0202     }
src/websocket/agent-gateway.ts:0203
src/websocket/agent-gateway.ts:0204     function getPayloadTenantId(payload: DashboardEventPayload): string | undefined {
src/websocket/agent-gateway.ts:0205       if (Array.isArray(payload)) {
src/websocket/agent-gateway.ts:0206         return undefined;
src/websocket/agent-gateway.ts:0207       }
src/websocket/agent-gateway.ts:0208
src/websocket/agent-gateway.ts:0209       return optionalString(payload.tenantId);
src/websocket/agent-gateway.ts:0210     }
src/websocket/agent-gateway.ts:0211
src/websocket/agent-gateway.ts:0212     function serializeDashboardPayload(payload: DashboardEventPayload): JsonObject | JsonObject[] {
src/websocket/agent-gateway.ts:0213       return Array.isArray(payload)
src/websocket/agent-gateway.ts:0214         ? payload.map((entry) => toJsonObject(entry))
src/websocket/agent-gateway.ts:0215         : toJsonObject(payload);
src/websocket/agent-gateway.ts:0216     }
src/websocket/agent-gateway.ts:0217
src/websocket/agent-gateway.ts:0218     function buildAgentListItem(agent: ConnectedAgent): AgentListItem {
src/websocket/agent-gateway.ts:0219       return {
src/websocket/agent-gateway.ts:0220         agentId: agent.agentId,
src/websocket/agent-gateway.ts:0221         hostname: agent.hostname,
src/websocket/agent-gateway.ts:0222         platform: agent.platform,
src/websocket/agent-gateway.ts:0223         architecture: agent.architecture,
src/websocket/agent-gateway.ts:0224         nodeKind: agent.nodeKind,
src/websocket/agent-gateway.ts:0225         deploymentClass: agent.deploymentClass,
src/websocket/agent-gateway.ts:0226         runtimeMode: agent.runtimeMode,
src/websocket/agent-gateway.ts:0227         tenantId: agent.tenantId,
src/websocket/agent-gateway.ts:0228         registration: agent.registration,
src/websocket/agent-gateway.ts:0229         lastHeartbeat: agent.lastHeartbeat.toISOString(),
src/websocket/agent-gateway.ts:0230         status: agent.status,
src/websocket/agent-gateway.ts:0231         isOnline: agent.status?.isOnline ?? true,
src/websocket/agent-gateway.ts:0232         synthetic: Boolean(agent.isSynthetic)
src/websocket/agent-gateway.ts:0233       };
src/websocket/agent-gateway.ts:0234     }
src/websocket/agent-gateway.ts:0235
src/websocket/agent-gateway.ts:0236     function isDetectionBootstrapError(error: unknown): boolean {
src/websocket/agent-gateway.ts:0237       const message = getErrorMessage(error).toLowerCase();
src/websocket/agent-gateway.ts:0238       return message.includes('database not initialized')
```

backend/src/websocket/agent-gateway.ts:239-288

```
src/websocket/agent-gateway.ts:0239         || message.includes('call initdatabase() first')
src/websocket/agent-gateway.ts:0240         || message.includes('detection pipeline storage not initialized');
src/websocket/agent-gateway.ts:0241     }
src/websocket/agent-gateway.ts:0242
src/websocket/agent-gateway.ts:0243     /**
src/websocket/agent-gateway.ts:0244      * SECURITY FIX CVE-2026-301: User-level rate limiting
src/websocket/agent-gateway.ts:0245      * Prevents bypass by opening multiple connections
src/websocket/agent-gateway.ts:0246      * Now uses Redis-backed rate limiter
src/websocket/agent-gateway.ts:0247      */
src/websocket/agent-gateway.ts:0248     async function checkUserRateLimit(userId: string, tenantId: string, socket: GatewaySocket): Promise<boolean> {
src/websocket/agent-gateway.ts:0249       const result = await socketRateLimitService.check({
src/websocket/agent-gateway.ts:0250         tenantId,
src/websocket/agent-gateway.ts:0251         key: `user:${userId}`,
src/websocket/agent-gateway.ts:0252         windowMs: USER_RATE_LIMIT_WINDOW,
src/websocket/agent-gateway.ts:0253         limit: MAX_USER_REQUESTS_PER_WINDOW,
src/websocket/agent-gateway.ts:0254         sourceIp: socket.handshake.address,
src/websocket/agent-gateway.ts:0255         sessionId: socket.id,
src/websocket/agent-gateway.ts:0256         headers: socket.handshake.headers as Record<string, unknown>,
src/websocket/agent-gateway.ts:0257         protocol: 'ws-agent'
src/websocket/agent-gateway.ts:0258       });
src/websocket/agent-gateway.ts:0259
src/websocket/agent-gateway.ts:0260       if (!result.allowed) {
src/websocket/agent-gateway.ts:0261         auditLogger.log({
src/websocket/agent-gateway.ts:0262           eventType: 'rate_limit_exceeded',
src/websocket/agent-gateway.ts:0263           category: 'auth',
src/websocket/agent-gateway.ts:0264           severity: 'warning',
src/websocket/agent-gateway.ts:0265           actor: { type: 'user', id: userId },
src/websocket/agent-gateway.ts:0266           action: 'rate_limit_check',
src/websocket/agent-gateway.ts:0267           details: {
src/websocket/agent-gateway.ts:0268             limit: MAX_USER_REQUESTS_PER_WINDOW,
src/websocket/agent-gateway.ts:0269             window: USER RATE LIMIT WINDOW,
src/websocket/agent-gateway.ts:0270             decisionSource: result.decisionSource,
src/websocket/agent-gateway.ts:0271             riskFlags: result.riskFlags
src/websocket/agent-gateway.ts:0272           },
src/websocket/agent-gateway.ts:0273           status: 'failure'
src/websocket/agent-gateway.ts:0274         });
src/websocket/agent-gateway.ts:0275       }
src/websocket/agent-gateway.ts:0276
src/websocket/agent-gateway.ts:0277       return result.allowed;
src/websocket/agent-gateway.ts:0278     }
src/websocket/agent-gateway.ts:0279
src/websocket/agent-gateway.ts:0280     /**
src/websocket/agent-gateway.ts:0281      * Socket-level rate limiting (for individual connections)
src/websocket/agent-gateway.ts:0282      */
src/websocket/agent-gateway.ts:0283     async function checkSocketRateLimit(socket: GatewaySocket, tenantId: string): Promise<boolean> {
src/websocket/agent-gateway.ts:0284       const result = await socketRateLimitService.check({
src/websocket/agent-gateway.ts:0285         tenantId,
src/websocket/agent-gateway.ts:0286         key: `socket:${socket.id}`,
src/websocket/agent-gateway.ts:0287         windowMs: RATE_LIMIT_WINDOW,
src/websocket/agent-gateway.ts:0288         limit: MAX_REQUESTS_PER_WINDOW,
```

backend/src/websocket/agent-gateway.ts:289-338

```
src/websocket/agent-gateway.ts:0289         sourceIp: socket.handshake.address,
src/websocket/agent-gateway.ts:0290         sessionId: socket.id,
src/websocket/agent-gateway.ts:0291         headers: socket.handshake.headers as Record<string, unknown>,
src/websocket/agent-gateway.ts:0292         protocol: 'ws-agent'
src/websocket/agent-gateway.ts:0293       });
src/websocket/agent-gateway.ts:0294       return result.allowed;
src/websocket/agent-gateway.ts:0295     }
src/websocket/agent-gateway.ts:0296
src/websocket/agent-gateway.ts:0297     /**
src/websocket/agent-gateway.ts:0298      * SECURITY FIX CVE-2026-300: Enhanced IP address validation
src/websocket/agent-gateway.ts:0299      * Uses Node's built-in net module for comprehensive validation
src/websocket/agent-gateway.ts:0300      */
src/websocket/agent-gateway.ts:0301     function isValidIPAddress(ip: string): boolean {
src/websocket/agent-gateway.ts:0302       if (!ip || typeof ip !== 'string') {
src/websocket/agent-gateway.ts:0303         return false;
src/websocket/agent-gateway.ts:0304       }
src/websocket/agent-gateway.ts:0305
src/websocket/agent-gateway.ts:0306       // Trim whitespace
src/websocket/agent-gateway.ts:0307       ip = ip.trim();
src/websocket/agent-gateway.ts:0308
src/websocket/agent-gateway.ts:0309       // Check for reasonable length limits
src/websocket/agent-gateway.ts:0310       if (ip.length > 45) { // Max IPv6 with zone index
src/websocket/agent-gateway.ts:0311         return false;
src/websocket/agent-gateway.ts:0312       }
src/websocket/agent-gateway.ts:0313
src/websocket/agent-gateway.ts:0314       // Handle IPv4-mapped IPv6 addresses (::ffff:192.0.2.1)
src/websocket/agent-gateway.ts:0315       if (ip.startsWith('::ffff:')) {
src/websocket/agent-gateway.ts:0316         const ipv4Part = ip.substring(7);
src/websocket/agent-gateway.ts:0317         return isIPv4(ipv4Part);
src/websocket/agent-gateway.ts:0318       }
src/websocket/agent-gateway.ts:0319
src/websocket/agent-gateway.ts:0320       // Use Node's built-in validation
src/websocket/agent-gateway.ts:0321       if (isIPv4(ip) || isIPv6(ip)) {
src/websocket/agent-gateway.ts:0322         // Additional check: prevent injection attempts
src/websocket/agent-gateway.ts:0323         // Reject if contains shell metacharacters or suspicious patterns
src/websocket/agent-gateway.ts:0324         const dangerousPatterns = [
src/websocket/agent-gateway.ts:0325           /\n/, /\r/, /\t/, // Control characters
src/websocket/agent-gateway.ts:0326           /\\$/, // Trailing backslash (escape attempt)
src/websocket/agent-gateway.ts:0327           /\$\(/, // Command substitution
src/websocket/agent-gateway.ts:0328           /`/, // Backtick command substitution
src/websocket/agent-gateway.ts:0329           /\|\s*\w/, // Pipe with command
src/websocket/agent-gateway.ts:0330         ];
src/websocket/agent-gateway.ts:0331
src/websocket/agent-gateway.ts:0332         for (const pattern of dangerousPatterns) {
src/websocket/agent-gateway.ts:0333           if (pattern.test(ip)) {
src/websocket/agent-gateway.ts:0334             logger.warn(`Suspicious pattern detected in IP address`, { ip });
src/websocket/agent-gateway.ts:0335             return false;
src/websocket/agent-gateway.ts:0336           }
src/websocket/agent-gateway.ts:0337         }
src/websocket/agent-gateway.ts:0338
```

backend/src/websocket/agent-gateway.ts:339-388

```
src/websocket/agent-gateway.ts:0339         return true;
src/websocket/agent-gateway.ts:0340       }
src/websocket/agent-gateway.ts:0341
src/websocket/agent-gateway.ts:0342       return false;
src/websocket/agent-gateway.ts:0343     }
src/websocket/agent-gateway.ts:0344
src/websocket/agent-gateway.ts:0345     /**
src/websocket/agent-gateway.ts:0346      * SECURITY FIX: Check for shell metacharacters
src/websocket/agent-gateway.ts:0347      */
src/websocket/agent-gateway.ts:0348     function containsShellMetacharacters(input: string): boolean {
src/websocket/agent-gateway.ts:0349       const dangerousChars = /[;&|`$(){}[\]<>'"!#%^*?\n\r\t\\]/;
src/websocket/agent-gateway.ts:0350       return dangerousChars.test(input);
src/websocket/agent-gateway.ts:0351     }
src/websocket/agent-gateway.ts:0352
src/websocket/agent-gateway.ts:0353     interface ConnectedAgent {
src/websocket/agent-gateway.ts:0354       socketId: string;
src/websocket/agent-gateway.ts:0355       agentId: string;
src/websocket/agent-gateway.ts:0356       hostname: string;
src/websocket/agent-gateway.ts:0357       platform: string;
src/websocket/agent-gateway.ts:0358       architecture?: string;
src/websocket/agent-gateway.ts:0359       nodeKind?: 'cloud' | 'edge' | 'vehicle';
src/websocket/agent-gateway.ts:0360       deploymentClass?: 'standard' | 'edge' | 'vehicle';
src/websocket/agent-gateway.ts:0361       runtimeMode?: 'normal' | 'read_only_bypass';
src/websocket/agent-gateway.ts:0362       tenantId: string;
src/websocket/agent-gateway.ts:0363       registration?: DashboardEventRecord;
src/websocket/agent-gateway.ts:0364       authenticated: boolean;
src/websocket/agent-gateway.ts:0365       lastHeartbeat: Date;
src/websocket/agent-gateway.ts:0366       status: AgentStatus | null;
src/websocket/agent-gateway.ts:0367       isSynthetic?: boolean;
src/websocket/agent-gateway.ts:0368       commandHandler?: (command: SyntheticAgentCommand) => boolean;
src/websocket/agent-gateway.ts:0369     }
src/websocket/agent-gateway.ts:0370
src/websocket/agent-gateway.ts:0371     interface DashboardClient {
src/websocket/agent-gateway.ts:0372       userId: string;
src/websocket/agent-gateway.ts:0373       socketId: string;
src/websocket/agent-gateway.ts:0374       tenantId: string;
src/websocket/agent-gateway.ts:0375     }
src/websocket/agent-gateway.ts:0376
src/websocket/agent-gateway.ts:0377     export class AgentGateway {
src/websocket/agent-gateway.ts:0378       private io: SocketIOServer<
src/websocket/agent-gateway.ts:0379         AgentClientToServerEvents,
src/websocket/agent-gateway.ts:0380         AgentServerToClientEvents,
src/websocket/agent-gateway.ts:0381         Record<string, never>,
src/websocket/agent-gateway.ts:0382         GatewaySocketData
src/websocket/agent-gateway.ts:0383       >;
src/websocket/agent-gateway.ts:0384       private agents: Map<string, ConnectedAgent> = new Map();
src/websocket/agent-gateway.ts:0385       private dashboardClients: Map<string, DashboardClient> = new Map();
src/websocket/agent-gateway.ts:0386       private testOnlineCounter = 0;
src/websocket/agent-gateway.ts:0387
src/websocket/agent-gateway.ts:0388       /**
```

backend/src/websocket/agent-gateway.ts:389-438

```
src/websocket/agent-gateway.ts:0389         * Broadcast audit log to dashboard clients
src/websocket/agent-gateway.ts:0390         */
src/websocket/agent-gateway.ts:0391        public broadcastAuditLog(log: DashboardEventRecord): void {
src/websocket/agent-gateway.ts:0392          this.broadcastToDashboards('audit:log', log, { tenantId: optionalString(log.tenantId) });
src/websocket/agent-gateway.ts:0393        }
src/websocket/agent-gateway.ts:0394
src/websocket/agent-gateway.ts:0395        constructor(httpServer: HttpServer) {
src/websocket/agent-gateway.ts:0396          this.io = new SocketIOServer(httpServer, {
src/websocket/agent-gateway.ts:0397            cors: {
src/websocket/agent-gateway.ts:0398              origin: process.env.ALLOWED ORIGINS?.split(',') || ['http://localhost:5173'],
src/websocket/agent-gateway.ts:0399              methods: ['GET', 'POST'],
src/websocket/agent-gateway.ts:0400              credentials: true
src/websocket/agent-gateway.ts:0401            },
src/websocket/agent-gateway.ts:0402            maxHttpBufferSize: 1e6 // 1MB
src/websocket/agent-gateway.ts:0403          });
src/websocket/agent-gateway.ts:0404
src/websocket/agent-gateway.ts:0405          this.setupAuthentication();
src/websocket/agent-gateway.ts:0406          this.setupEventHandlers();
src/websocket/agent-gateway.ts:0407
src/websocket/agent-gateway.ts:0408          logger.info('Agent Gateway initialized with secure authentication');
src/websocket/agent-gateway.ts:0409        }
src/websocket/agent-gateway.ts:0410
src/websocket/agent-gateway.ts:0411        /**
src/websocket/agent-gateway.ts:0412         * SECURITY FIX: Enhanced authentication middleware
src/websocket/agent-gateway.ts:0413         */
src/websocket/agent-gateway.ts:0414        private setupAuthentication(): void {
src/websocket/agent-gateway.ts:0415          this.io.use(async (socket, next) => {
src/websocket/agent-gateway.ts:0416            const auth = getHandshakeAuth(socket);
src/websocket/agent-gateway.ts:0417            const clientType: GatewayClientType = optionalString(auth.type) === 'agent' ? 'agent' : 'dashboard';
src/websocket/agent-gateway.ts:0418            const clientIp = socket.handshake.address;
src/websocket/agent-gateway.ts:0419
src/websocket/agent-gateway.ts:0420            try {
src/websocket/agent-gateway.ts:0421              if (isTestRuntime()) {
src/websocket/agent-gateway.ts:0422                if (clientType === 'agent') {
src/websocket/agent-gateway.ts:0423                  const agent = wsTestAgentAdapter(auth, socket.id) as GatewayAuthenticatedAgent;
src/websocket/agent-gateway.ts:0424                  socket.data.agent = agent;
src/websocket/agent-gateway.ts:0425                  socket.data.clientType = 'agent';
src/websocket/agent-gateway.ts:0426                  socket.data.tenantContext =
src/websocket/agent-gateway.ts:0427                    resolveAgentTenantContext(agent.tenantId, 'test-agent') ||
src/websocket/agent-gateway.ts:0428                    buildPlatformTenantContext('test-agent');
src/websocket/agent-gateway.ts:0429                  return next();
src/websocket/agent-gateway.ts:0430                }
src/websocket/agent-gateway.ts:0431
src/websocket/agent-gateway.ts:0432                const user = wsTestDashboardUserAdapter(auth);
src/websocket/agent-gateway.ts:0433                socket.data.user = user;
src/websocket/agent-gateway.ts:0434                socket.data.clientType = 'dashboard';
src/websocket/agent-gateway.ts:0435                socket.data.tenantContext = resolveUserTenantContext(user, {
src/websocket/agent-gateway.ts:0436                  allowAdminPlatformFallback: user.role === 'admin'
src/websocket/agent-gateway.ts:0437                }) || buildPlatformTenantContext('test-dashboard');
src/websocket/agent-gateway.ts:0438                return next();
```

backend/src/websocket/agent-gateway.ts:439-488

```
src/websocket/agent-gateway.ts:0439             }
src/websocket/agent-gateway.ts:0440
src/websocket/agent-gateway.ts:0441             // Agent authentication
src/websocket/agent-gateway.ts:0442             if (clientType === 'agent') {
src/websocket/agent-gateway.ts:0443               const agentId = optionalString(auth.agentId);
src/websocket/agent-gateway.ts:0444               const apiKey = optionalString(auth.apiKey);
src/websocket/agent-gateway.ts:0445
src/websocket/agent-gateway.ts:0446               if (!agentId || !apiKey) {
src/websocket/agent-gateway.ts:0447                 await auditLogger.logAgent({
src/websocket/agent-gateway.ts:0448                   action: 'connect',
src/websocket/agent-gateway.ts:0449                   agentId: agentId || 'unknown',
src/websocket/agent-gateway.ts:0450                   ip: clientIp,
src/websocket/agent-gateway.ts:0451                   success: false,
src/websocket/agent-gateway.ts:0452                   details: { reason: 'Missing credentials' }
src/websocket/agent-gateway.ts:0453                 });
src/websocket/agent-gateway.ts:0454                 return next(new Error('backend.agent_id_and_api_key_required'));
src/websocket/agent-gateway.ts:0455               }
src/websocket/agent-gateway.ts:0456
src/websocket/agent-gateway.ts:0457               // Verify agent against database
src/websocket/agent-gateway.ts:0458               const authenticatedAgent = await agentAuthService.verifyAgent(agentId, apiKey);
src/websocket/agent-gateway.ts:0459
src/websocket/agent-gateway.ts:0460               if (!authenticatedAgent) {
src/websocket/agent-gateway.ts:0461                 await auditLogger.logAgent({
src/websocket/agent-gateway.ts:0462                   action: 'connect',
src/websocket/agent-gateway.ts:0463                   agentId,
src/websocket/agent-gateway.ts:0464                   ip: clientIp,
src/websocket/agent-gateway.ts:0465                   success: false,
src/websocket/agent-gateway.ts:0466                   details: { reason: 'Invalid credentials' }
src/websocket/agent-gateway.ts:0467                 });
src/websocket/agent-gateway.ts:0468                 return next(new Error('backend.invalid_agent_credentials'));
src/websocket/agent-gateway.ts:0469               }
src/websocket/agent-gateway.ts:0470
src/websocket/agent-gateway.ts:0471               // Attach to socket
src/websocket/agent-gateway.ts:0472               const tenantContext = resolveAgentTenantContext(authenticatedAgent.tenantId, 'agent-auth');
src/websocket/agent-gateway.ts:0473               if (!tenantContext) {
src/websocket/agent-gateway.ts:0474                 await auditLogger.logAgent({
src/websocket/agent-gateway.ts:0475                   action: 'connect',
src/websocket/agent-gateway.ts:0476                   agentId,
src/websocket/agent-gateway.ts:0477                   ip: clientIp,
src/websocket/agent-gateway.ts:0478                   success: false,
src/websocket/agent-gateway.ts:0479                   details: { reason: 'tenant_not_resolved' }
src/websocket/agent-gateway.ts:0480                 });
src/websocket/agent-gateway.ts:0481                 return next(new Error('backend.agent tenant not resolved'));
src/websocket/agent-gateway.ts:0482               }
src/websocket/agent-gateway.ts:0483
src/websocket/agent-gateway.ts:0484               socket.data.agent = authenticatedAgent;
src/websocket/agent-gateway.ts:0485               socket.data.clientType = 'agent';
src/websocket/agent-gateway.ts:0486               socket.data.tenantContext = tenantContext;
src/websocket/agent-gateway.ts:0487
src/websocket/agent-gateway.ts:0488               await auditLogger.logAgent({
```

backend/src/websocket/agent-gateway.ts:489-538

```
src/websocket/agent-gateway.ts:0489                action: 'connect',
src/websocket/agent-gateway.ts:0490                agentId: authenticatedAgent.agentId,
src/websocket/agent-gateway.ts:0491                ip: clientIp,
src/websocket/agent-gateway.ts:0492                success: true,
src/websocket/agent-gateway.ts:0493                details: { hostname: authenticatedAgent.hostname }
src/websocket/agent-gateway.ts:0494              });
src/websocket/agent-gateway.ts:0495
src/websocket/agent-gateway.ts:0496              return next();
src/websocket/agent-gateway.ts:0497            }
src/websocket/agent-gateway.ts:0498
src/websocket/agent-gateway.ts:0499            // Dashboard authentication
src/websocket/agent-gateway.ts:0500            // Try to get token from auth.token first, then from cookies
src/websocket/agent-gateway.ts:0501            let token = optionalString(auth.token);
src/websocket/agent-gateway.ts:0502
src/websocket/agent-gateway.ts:0503            // If no token in auth, try to get from cookie
src/websocket/agent-gateway.ts:0504            const cookieHeader = getCookieHeader(socket.handshake.headers.cookie);
src/websocket/agent-gateway.ts:0505            if (!token && cookieHeader) {
src/websocket/agent-gateway.ts:0506              const cookies = cookieHeader;
src/websocket/agent-gateway.ts:0507              // Parse cookies - handle multiple formats
src/websocket/agent-gateway.ts:0508              const cookiePairs = cookies.split(';').map(c => c.trim());
src/websocket/agent-gateway.ts:0509              for (const pair of cookiePairs) {
src/websocket/agent-gateway.ts:0510                const [name, value] = pair.split('=');
src/websocket/agent-gateway.ts:0511                if (name === 'accessToken' && value) {
src/websocket/agent-gateway.ts:0512                  token = value;
src/websocket/agent-gateway.ts:0513                  break;
src/websocket/agent-gateway.ts:0514                }
src/websocket/agent-gateway.ts:0515              }
src/websocket/agent-gateway.ts:0516            }
src/websocket/agent-gateway.ts:0517
src/websocket/agent-gateway.ts:0518            if (!token) {
src/websocket/agent-gateway.ts:0519              await auditLogger.log({
src/websocket/agent-gateway.ts:0520                eventType: 'dashboard connection',
src/websocket/agent-gateway.ts:0521                category: 'auth',
src/websocket/agent-gateway.ts:0522                severity: 'warning',
src/websocket/agent-gateway.ts:0523                actor: { type: 'user', id: 'unknown', ip: clientIp },
src/websocket/agent-gateway.ts:0524                action: 'connect',
src/websocket/agent-gateway.ts:0525                details: { reason: 'No token provided', cookies: cookieHeader },
src/websocket/agent-gateway.ts:0526                status: 'failure'
src/websocket/agent-gateway.ts:0527              });
src/websocket/agent-gateway.ts:0528              return next(new Error('backend.authentication_token_required'));
src/websocket/agent-gateway.ts:0529            }
src/websocket/agent-gateway.ts:0530
src/websocket/agent-gateway.ts:0531            // Verify JWT
src/websocket/agent-gateway.ts:0532            const user = await verifyTokenAsync(token);
src/websocket/agent-gateway.ts:0533
src/websocket/agent-gateway.ts:0534            if (!user) {
src/websocket/agent-gateway.ts:0535              await auditLogger.log({
src/websocket/agent-gateway.ts:0536                eventType: 'dashboard_connection',
src/websocket/agent-gateway.ts:0537                category: 'auth',
src/websocket/agent-gateway.ts:0538                severity: 'warning',
```

backend/src/websocket/agent-gateway.ts:539-588

```
src/websocket/agent-gateway.ts:0539                actor: { type: 'user', id: 'unknown', ip: clientIp },
src/websocket/agent-gateway.ts:0540                action: 'connect',
src/websocket/agent-gateway.ts:0541                details: { reason: 'Invalid token' },
src/websocket/agent-gateway.ts:0542                status: 'failure'
src/websocket/agent-gateway.ts:0543              });
src/websocket/agent-gateway.ts:0544              return next(new Error('backend.invalid_or_expired_token'));
src/websocket/agent-gateway.ts:0545            }
src/websocket/agent-gateway.ts:0546
src/websocket/agent-gateway.ts:0547            // Attach to socket
src/websocket/agent-gateway.ts:0548            const tenantContext = resolveUserTenantContext(user, {
src/websocket/agent-gateway.ts:0549              allowAdminPlatformFallback: user.role === 'admin'
src/websocket/agent-gateway.ts:0550            });
src/websocket/agent-gateway.ts:0551            if (!tenantContext) {
src/websocket/agent-gateway.ts:0552              await auditLogger.log({
src/websocket/agent-gateway.ts:0553                eventType: 'dashboard_connection',
src/websocket/agent-gateway.ts:0554                category: 'auth',
src/websocket/agent-gateway.ts:0555                severity: 'warning',
src/websocket/agent-gateway.ts:0556                actor: { type: 'user', id: user.userId, ip: clientIp },
src/websocket/agent-gateway.ts:0557                action: 'connect',
src/websocket/agent-gateway.ts:0558                details: { reason: 'tenant not resolved', email: user.email },
src/websocket/agent-gateway.ts:0559                status: 'failure'
src/websocket/agent-gateway.ts:0560              });
src/websocket/agent-gateway.ts:0561              return next(new Error('backend.dashboard_tenant_not_resolved'));
src/websocket/agent-gateway.ts:0562            }
src/websocket/agent-gateway.ts:0563
src/websocket/agent-gateway.ts:0564            socket.data.user = user;
src/websocket/agent-gateway.ts:0565            socket.data.clientType = 'dashboard';
src/websocket/agent-gateway.ts:0566            socket.data.tenantContext = tenantContext;
src/websocket/agent-gateway.ts:0567
src/websocket/agent-gateway.ts:0568            await auditLogger.log({
src/websocket/agent-gateway.ts:0569              eventType: 'dashboard connection',
src/websocket/agent-gateway.ts:0570              category: 'auth',
src/websocket/agent-gateway.ts:0571              severity: 'info',
src/websocket/agent-gateway.ts:0572              actor: { type: 'user', id: user.userId, ip: clientIp },
src/websocket/agent-gateway.ts:0573              action: 'connect',
src/websocket/agent-gateway.ts:0574              details: { email: user.email },
src/websocket/agent-gateway.ts:0575              status: 'success'
src/websocket/agent-gateway.ts:0576            });
src/websocket/agent-gateway.ts:0577
src/websocket/agent-gateway.ts:0578            next();
src/websocket/agent-gateway.ts:0579          } catch (error) {
src/websocket/agent-gateway.ts:0580            logger.error('Authentication error', { error: getErrorMessage(error), clientIp });
src/websocket/agent-gateway.ts:0581            return next(new Error('backend.authentication failed'));
src/websocket/agent-gateway.ts:0582          }
src/websocket/agent-gateway.ts:0583        });
src/websocket/agent-gateway.ts:0584      }
src/websocket/agent-gateway.ts:0585
src/websocket/agent-gateway.ts:0586      /**
src/websocket/agent-gateway.ts:0587       * Setup event handlers
src/websocket/agent-gateway.ts:0588       */
```

backend/src/websocket/agent-gateway.ts:589-638

```
src/websocket/agent-gateway.ts:0589       private setupEventHandlers(): void {
src/websocket/agent-gateway.ts:0590         this.io.on('connection', (socket: GatewaySocket) => {
src/websocket/agent-gateway.ts:0591           const clientType = socket.data.clientType;
src/websocket/agent-gateway.ts:0592
src/websocket/agent-gateway.ts:0593           logger.info(`Client connected: ${socket.id} (${clientType})`);
src/websocket/agent-gateway.ts:0594
src/websocket/agent-gateway.ts:0595           if (clientType === 'agent') {
src/websocket/agent-gateway.ts:0596             this.handleAgentConnection(socket);
src/websocket/agent-gateway.ts:0597           } else {
src/websocket/agent-gateway.ts:0598             this.handleDashboardConnection(socket);
src/websocket/agent-gateway.ts:0599           }
src/websocket/agent-gateway.ts:0600
src/websocket/agent-gateway.ts:0601           socket.on('disconnect', () => this.handleDisconnect(socket));
src/websocket/agent-gateway.ts:0602         });
src/websocket/agent-gateway.ts:0603       }
src/websocket/agent-gateway.ts:0604
src/websocket/agent-gateway.ts:0605       /**
src/websocket/agent-gateway.ts:0606        * SECURITY FIX: Enhanced agent connection handler
src/websocket/agent-gateway.ts:0607        */
src/websocket/agent-gateway.ts:0608       private handleAgentConnection(socket: GatewaySocket): void {
src/websocket/agent-gateway.ts:0609         const agent = socket.data.agent;
src/websocket/agent-gateway.ts:0610         const tenantContext = socket.data.tenantContext;
src/websocket/agent-gateway.ts:0611         if (!agent || !tenantContext) {
src/websocket/agent-gateway.ts:0612           socket.emit('error', { message: 'backend.agent_context_not_available', reason: 'tenant_required' });
src/websocket/agent-gateway.ts:0613           socket.disconnect();
src/websocket/agent-gateway.ts:0614           return;
src/websocket/agent-gateway.ts:0615         }
src/websocket/agent-gateway.ts:0616
src/websocket/agent-gateway.ts:0617         // Store connected agent
src/websocket/agent-gateway.ts:0618         const connectedAgent: ConnectedAgent = {
src/websocket/agent-gateway.ts:0619           socketId: socket.id,
src/websocket/agent-gateway.ts:0620           agentId: agent.agentId,
src/websocket/agent-gateway.ts:0621           hostname: agent.hostname,
src/websocket/agent-gateway.ts:0622           platform: agent.platform,
src/websocket/agent-gateway.ts:0623           architecture: agent.architecture || agent.arch || undefined,
src/websocket/agent-gateway.ts:0624           nodeKind: agent.nodeKind || undefined,
src/websocket/agent-gateway.ts:0625           deploymentClass: agent.deploymentClass || undefined,
src/websocket/agent-gateway.ts:0626           runtimeMode: 'normal',
src/websocket/agent-gateway.ts:0627           tenantId: tenantContext.tenantId,
src/websocket/agent-gateway.ts:0628           authenticated: true,
src/websocket/agent-gateway.ts:0629           lastHeartbeat: new Date(),
src/websocket/agent-gateway.ts:0630           status: null
src/websocket/agent-gateway.ts:0631         };
src/websocket/agent-gateway.ts:0632
src/websocket/agent-gateway.ts:0633         this.agents.set(agent.agentId, connectedAgent);
src/websocket/agent-gateway.ts:0634         if (isTestRuntime()) {
src/websocket/agent-gateway.ts:0635           this.testOnlineCounter += 1;
src/websocket/agent-gateway.ts:0636         }
src/websocket/agent-gateway.ts:0637         this.broadcastAgentListUpdates(tenantContext.tenantId);
src/websocket/agent-gateway.ts:0638
```

backend/src/websocket/agent-gateway.ts:639-688

```
src/websocket/agent-gateway.ts:0639         // Send confirmation to agent
src/websocket/agent-gateway.ts:0640         socket.emit('agent:authenticated', {
src/websocket/agent-gateway.ts:0641           agentId: agent.agentId,
src/websocket/agent-gateway.ts:0642           timestamp: Date.now()
src/websocket/agent-gateway.ts:0643         });
src/websocket/agent-gateway.ts:0644
src/websocket/agent-gateway.ts:0645         // Broadcast to dashboards
src/websocket/agent-gateway.ts:0646         this.broadcastToDashboards('agent:online', {
src/websocket/agent-gateway.ts:0647           agentId: agent.agentId,
src/websocket/agent-gateway.ts:0648           hostname: agent.hostname,
src/websocket/agent-gateway.ts:0649           platform: agent.platform,
src/websocket/agent-gateway.ts:0650           tenantId: tenantContext.tenantId
src/websocket/agent-gateway.ts:0651         }, { tenantId: tenantContext.tenantId });
src/websocket/agent-gateway.ts:0652
src/websocket/agent-gateway.ts:0653         void this.syncDetectionRulesToAgent(agent.agentId, tenantContext.tenantId);
src/websocket/agent-gateway.ts:0654
src/websocket/agent-gateway.ts:0655         // Legacy registration event used by tests and older agents.
src/websocket/agent-gateway.ts:0656         socket.on('agent:register', (registration) => {
src/websocket/agent-gateway.ts:0657           const connectedAgent = this.agents.get(agent.agentId);
src/websocket/agent-gateway.ts:0658           if (!connectedAgent) return;
src/websocket/agent-gateway.ts:0659           const validatedRegistration = validateAgentRegisterPayload(registration);
src/websocket/agent-gateway.ts:0660           if (!validatedRegistration) {
src/websocket/agent-gateway.ts:0661             socket.emit('error', { message: 'backend.invalid_agent_registration_payload', reason: 'invalid_payload' });
src/websocket/agent-gateway.ts:0662             return;
src/websocket/agent-gateway.ts:0663           }
src/websocket/agent-gateway.ts:0664
src/websocket/agent-gateway.ts:0665           connectedAgent.registration = { ...validatedRegistration };
src/websocket/agent-gateway.ts:0666           connectedAgent.hostname = validatedRegistration.hostname || connectedAgent.hostname;
src/websocket/agent-gateway.ts:0667           connectedAgent.platform = validatedRegistration.platform || connectedAgent.platform;
src/websocket/agent-gateway.ts:0668           connectedAgent.architecture = validatedRegistration.architecture || validatedRegistration.arch || connectedAgent.architecture;
src/websocket/agent-gateway.ts:0669           connectedAgent.nodeKind = validatedRegistration.nodeKind || connectedAgent.nodeKind;
src/websocket/agent-gateway.ts:0670           connectedAgent.deploymentClass = validatedRegistration.deploymentClass || connectedAgent.deploymentClass;
src/websocket/agent-gateway.ts:0671           connectedAgent.lastHeartbeat = new Date();
src/websocket/agent-gateway.ts:0672
src/websocket/agent-gateway.ts:0673           void edgeAutoService.upsertUsageSnapshot(connectedAgent.tenantId, connectedAgent.agentId, {
src/websocket/agent-gateway.ts:0674             nodeKind: connectedAgent.nodeKind,
src/websocket/agent-gateway.ts:0675             architecture: connectedAgent.architecture,
src/websocket/agent-gateway.ts:0676             deploymentClass: connectedAgent.deploymentClass,
src/websocket/agent-gateway.ts:0677           });
src/websocket/agent-gateway.ts:0678
src/websocket/agent-gateway.ts:0679           this.broadcastToDashboards('agent:registered', {
src/websocket/agent-gateway.ts:0680             agentId: connectedAgent.agentId,
src/websocket/agent-gateway.ts:0681             registration: connectedAgent.registration,
src/websocket/agent-gateway.ts:0682             tenantId: connectedAgent.tenantId
src/websocket/agent-gateway.ts:0683           }, { tenantId: connectedAgent.tenantId });
src/websocket/agent-gateway.ts:0684         });
src/websocket/agent-gateway.ts:0685
src/websocket/agent-gateway.ts:0686         // Heartbeat handler
src/websocket/agent-gateway.ts:0687         // SECURITY FIX CVE-2026-201: Validate heartbeat comes from authenticated agent
src/websocket/agent-gateway.ts:0688         socket.on('agent:heartbeat', async (status) => {
```

backend/src/websocket/agent-gateway.ts:689-700 | backend/src/api/routes/devices.routes.ts:1-38

```
src/websocket/agent-gateway.ts:0689           const validatedStatus = validateAgentHeartbeatPayload(status);
src/websocket/agent-gateway.ts:0690           if (!validatedStatus) {
src/websocket/agent-gateway.ts:0691             socket.emit('error', { message: 'backend.invalid_agent_heartbeat_payload', reason: 'invalid_payload' });
src/websocket/agent-gateway.ts:0692             return;
src/websocket/agent-gateway.ts:0693           }
src/websocket/agent-gateway.ts:0694
src/websocket/agent-gateway.ts:0695           // SECURITY FIX: Verify heartbeat agentId matches authenticated agent
src/websocket/agent-gateway.ts:0696           if (validatedStatus.agentId && validatedStatus.agentId !== agent.agentId) {
src/websocket/agent-gateway.ts:0697             logger.warn('Heartbeat agent ID mismatch', {
src/websocket/agent-gateway.ts:0698               socketId: socket.id,
src/websocket/agent-gateway.ts:0699               expected: agent.agentId,
src/websocket/agent-gateway.ts:0700               received: validatedStatus.agentId
src/api/routes/devices.routes.ts:0001   /**
src/api/routes/devices.routes.ts:0002    * AetherGuard - ESP32 Device Management API
src/api/routes/devices.routes.ts:0003    *
src/api/routes/devices.routes.ts:0004    * REST API for ESP32 firmware gateway
src/api/routes/devices.routes.ts:0005    * SECURITY FIX CVE-2026-031: Added authentication middleware
src/api/routes/devices.routes.ts:0006    * SECURITY FIX: Route ordering - static routes before dynamic routes
src/api/routes/devices.routes.ts:0007    */
src/api/routes/devices.routes.ts:0008
src/api/routes/devices.routes.ts:0009   import { Router, Request, Response } from 'express';
src/api/routes/devices.routes.ts:0010   import { authenticate, requireRole, AuthenticatedRequest } from '../../middleware/auth.middleware';
src/api/routes/devices.routes.ts:0011   import logger from '../../utils/logger';
src/api/routes/devices.routes.ts:0012   import { securityOperationsService } from '../../services/security-operations.service';
src/api/routes/devices.routes.ts:0013   import auditLogger from '../../security/audit-logger.service';
src/api/routes/devices.routes.ts:0014   import type { AuditEvent } from '../../security/audit-logger.service';
src/api/routes/devices.routes.ts:0015   import { commandDispatcherService } from '../../services/command-dispatcher.service';
src/api/routes/devices.routes.ts:0016   import { PLATFORM_TENANT_ID, resolveRequestTenantContext } from '../../core/tenant/tenant-context';
src/api/routes/devices.routes.ts:0017   import type { JsonObject } from '../../types/json';
src/api/routes/devices.routes.ts:0018   import { asRecord, getBoolean, getErrorMessage, getString, getStringArray } from '../../utils/typed-access';
src/api/routes/devices.routes.ts:0019
src/api/routes/devices.routes.ts:0020   const router = Router();
src/api/routes/devices.routes.ts:0021
src/api/routes/devices.routes.ts:0022   // SECURITY FIX CVE-2026-031: Apply authentication to all routes
src/api/routes/devices.routes.ts:0023   router.use(authenticate);
src/api/routes/devices.routes.ts:0024
src/api/routes/devices.routes.ts:0025   // Device storage (in-memory + can be extended to database)
src/api/routes/devices.routes.ts:0026   interface DeviceInfo {
src/api/routes/devices.routes.ts:0027       deviceId: string;
src/api/routes/devices.routes.ts:0028       version: string;
src/api/routes/devices.routes.ts:0029       chipModel: string;
src/api/routes/devices.routes.ts:0030       flashSize: number;
src/api/routes/devices.routes.ts:0031       freeHeap: number;
src/api/routes/devices.routes.ts:0032       sdkVersion: string;
src/api/routes/devices.routes.ts:0033       mode: 'online' | 'offline';
src/api/routes/devices.routes.ts:0034       isOnline: boolean;
src/api/routes/devices.routes.ts:0035       lastHeartbeat: Date;
src/api/routes/devices.routes.ts:0036       connectedAgents: string[];
src/api/routes/devices.routes.ts:0037       status?: {
src/api/routes/devices.routes.ts:0038           cpuUsage: number;
```

backend/src/api/routes/devices.routes.ts:39-88

```
src/api/routes/devices.routes.ts:0039           memoryUsage: number;
src/api/routes/devices.routes.ts:0040           uptime: number;
src/api/routes/devices.routes.ts:0041           wifiRssi: number;
src/api/routes/devices.routes.ts:0042       };
src/api/routes/devices.routes.ts:0043       systemInfo?: {
src/api/routes/devices.routes.ts:0044           hostname: string;
src/api/routes/devices.routes.ts:0045           os: string;
src/api/routes/devices.routes.ts:0046           kernel: string;
src/api/routes/devices.routes.ts:0047           arch: string;
src/api/routes/devices.routes.ts:0048       };
src/api/routes/devices.routes.ts:0049       transport?: 'wifi' | 'usb_serial' | 'ethernet' | 'unknown';
src/api/routes/devices.routes.ts:0050       externalPowerPresent?: boolean;
src/api/routes/devices.routes.ts:0051       secureElementPresent?: boolean;
src/api/routes/devices.routes.ts:0052       pendingPhysicalApproval?: boolean;
src/api/routes/devices.routes.ts:0053       interlockState?: 'disabled' | 'armed' | 'asserted' | 'fault' | 'unknown';
src/api/routes/devices.routes.ts:0054       hostActionState?: 'idle' | 'running' | 'awaiting_confirmation' | 'completed' | 'failed' | 'unknown';
src/api/routes/devices.routes.ts:0055       physicalAuth?: {
src/api/routes/devices.routes.ts:0056           publicKeyPem?: string;
src/api/routes/devices.routes.ts:0057           monotonicCounter?: number;
src/api/routes/devices.routes.ts:0058           lastConfirmedAt?: string;
src/api/routes/devices.routes.ts:0059       };
src/api/routes/devices.routes.ts:0060   }
src/api/routes/devices.routes.ts:0061
src/api/routes/devices.routes.ts:0062   interface DeviceConfig {
src/api/routes/devices.routes.ts:0063       displayName: string;
src/api/routes/devices.routes.ts:0064       group: string;
src/api/routes/devices.routes.ts:0065       site: string;
src/api/routes/devices.routes.ts:0066       networkProfile: string;
src/api/routes/devices.routes.ts:0067       scanProfile: string;
src/api/routes/devices.routes.ts:0068       notes: string;
src/api/routes/devices.routes.ts:0069       commandPermissions: string[];
src/api/routes/devices.routes.ts:0070       tags: string[];
src/api/routes/devices.routes.ts:0071       updatedAt: string;
src/api/routes/devices.routes.ts:0072   }
src/api/routes/devices.routes.ts:0073
src/api/routes/devices.routes.ts:0074   interface DeviceCommandRecord {
src/api/routes/devices.routes.ts:0075       id: string;
src/api/routes/devices.routes.ts:0076       deviceId: string;
src/api/routes/devices.routes.ts:0077       command: string;
src/api/routes/devices.routes.ts:0078       status: 'queued' | 'completed' | 'failed';
src/api/routes/devices.routes.ts:0079       message: string;
src/api/routes/devices.routes.ts:0080       executedBy: string;
src/api/routes/devices.routes.ts:0081       createdAt: string;
src/api/routes/devices.routes.ts:0082       completedAt?: string;
src/api/routes/devices.routes.ts:0083       params?: Record<string, unknown>;
src/api/routes/devices.routes.ts:0084   }
src/api/routes/devices.routes.ts:0085
src/api/routes/devices.routes.ts:0086   interface DeviceEvent {
src/api/routes/devices.routes.ts:0087       id: string;
src/api/routes/devices.routes.ts:0088       deviceId: string;
```

backend/src/api/routes/devices.routes.ts:89-138

```
src/api/routes/devices.routes.ts:0089       type: 'register' | 'heartbeat' | 'data' | 'alert' | 'unknown_error' | 'disconnect' | 'command';
src/api/routes/devices.routes.ts:0090       payload: unknown;
src/api/routes/devices.routes.ts:0091       timestamp: string;
src/api/routes/devices.routes.ts:0092   }
src/api/routes/devices.routes.ts:0093
src/api/routes/devices.routes.ts:0094   type AuditLogPayload = Omit<AuditEvent, 'id' | 'timestamp'>;
src/api/routes/devices.routes.ts:0095   type DeviceCommandParams = JsonObject;
src/api/routes/devices.routes.ts:0096   type LatestConfirmationSession = Record<string, unknown> | null;
src/api/routes/devices.routes.ts:0097
src/api/routes/devices.routes.ts:0098   const devices = new Map<string, DeviceInfo>();
src/api/routes/devices.routes.ts:0099   const deviceEvents = new Map<string, DeviceEvent[]>();
src/api/routes/devices.routes.ts:0100   const deviceConfigs = new Map<string, DeviceConfig>();
src/api/routes/devices.routes.ts:0101   const deviceCommands = new Map<string, DeviceCommandRecord[]>();
src/api/routes/devices.routes.ts:0102   let deviceCommandTransport: ((deviceId: string, command: string, params?: DeviceCommandParams) => Promise<boolean> | boolean) | null = null;
src/api/routes/devices.routes.ts:0103
src/api/routes/devices.routes.ts:0104   const ALL_DEVICE_COMMANDS = [
src/api/routes/devices.routes.ts:0105       'collect',
src/api/routes/devices.routes.ts:0106       'sync-config',
src/api/routes/devices.routes.ts:0107       'ota_update',
src/api/routes/devices.routes.ts:0108       'host backup',
src/api/routes/devices.routes.ts:0109       'host install',
src/api/routes/devices.routes.ts:0110       'reboot',
src/api/routes/devices.routes.ts:0111       'host_remove',
src/api/routes/devices.routes.ts:0112       'recovery_flash',
src/api/routes/devices.routes.ts:0113       'factory_reset',
src/api/routes/devices.routes.ts:0114       'diagnostic_unlock',
src/api/routes/devices.routes.ts:0115       'vehicle_ota_flash',
src/api/routes/devices.routes.ts:0116       'cellular_breaker_assert',
src/api/routes/devices.routes.ts:0117       'cellular_breaker_release'
src/api/routes/devices.routes.ts:0118   ];
src/api/routes/devices.routes.ts:0119
src/api/routes/devices.routes.ts:0120   function resolveActorTenantId(req?: AuthenticatedRequest): string {
src/api/routes/devices.routes.ts:0121       if (!req) {
src/api/routes/devices.routes.ts:0122           return PLATFORM_TENANT_ID;
src/api/routes/devices.routes.ts:0123       }
src/api/routes/devices.routes.ts:0124
src/api/routes/devices.routes.ts:0125       const context = resolveRequestTenantContext(req, {
src/api/routes/devices.routes.ts:0126           allowAdminPlatformFallback: req.user?.role === 'admin'
src/api/routes/devices.routes.ts:0127       });
src/api/routes/devices.routes.ts:0128
src/api/routes/devices.routes.ts:0129       return context?.tenantId || req.user?.tenantId || PLATFORM_TENANT_ID;
src/api/routes/devices.routes.ts:0130   }
src/api/routes/devices.routes.ts:0131
src/api/routes/devices.routes.ts:0132   function safeAuditLog(payload: AuditLogPayload): void {
src/api/routes/devices.routes.ts:0133       Promise.resolve(auditLogger.log(payload)).catch((error) => {
src/api/routes/devices.routes.ts:0134           logger.warn('Device audit log unavailable', { error: getErrorMessage(error) });
src/api/routes/devices.routes.ts:0135       });
src/api/routes/devices.routes.ts:0136   }
src/api/routes/devices.routes.ts:0137
src/api/routes/devices.routes.ts:0138   // Codebook
```

backend/src/api/routes/devices.routes.ts:139-188

```
src/api/routes/devices.routes.ts:0139   const codebook: Record<string, string> = {
src/api/routes/devices.routes.ts:0140       'E001': 'Connection refused',
src/api/routes/devices.routes.ts:0141       'E002': 'Permission denied',
src/api/routes/devices.routes.ts:0142       'E003': 'Out of memory',
src/api/routes/devices.routes.ts:0143       'E004': 'Disk full',
src/api/routes/devices.routes.ts:0144       'E005': 'Service unavailable',
src/api/routes/devices.routes.ts:0145       'S001': 'CPU usage critical (>90%)',
src/api/routes/devices.routes.ts:0146       'S002': 'Memory usage critical (>85%)',
src/api/routes/devices.routes.ts:0147       'S003': 'Disk usage critical (>90%)',
src/api/routes/devices.routes.ts:0148       'A001': 'SQL Injection detected',
src/api/routes/devices.routes.ts:0149       'A002': 'XSS attempt blocked',
src/api/routes/devices.routes.ts:0150       'A003': 'Path traversal attempt',
src/api/routes/devices.routes.ts:0151       'A004': 'Command injection detected',
src/api/routes/devices.routes.ts:0152       'A005': 'Brute force attack',
src/api/routes/devices.routes.ts:0153       'A006': 'DDoS pattern detected',
src/api/routes/devices.routes.ts:0154   };
src/api/routes/devices.routes.ts:0155
src/api/routes/devices.routes.ts:0156   function buildDefaultConfig(device: DeviceInfo): DeviceConfig {
src/api/routes/devices.routes.ts:0157       return {
src/api/routes/devices.routes.ts:0158           displayName: device.deviceId,
src/api/routes/devices.routes.ts:0159           group: device.mode === 'online' ? 'core-edge' : 'field-edge',
src/api/routes/devices.routes.ts:0160           site: device.systemInfo?.hostname || 'unassigned-site',
src/api/routes/devices.routes.ts:0161           networkProfile: device.mode === 'online' ? 'cloud-mesh' : 'standalone-buffer',
src/api/routes/devices.routes.ts:0162           scanProfile: device.isOnline ? 'balanced' : 'offline-recovery',
src/api/routes/devices.routes.ts:0163           notes: `${device.chipModel} running firmware ${device.version}`,
src/api/routes/devices.routes.ts:0164           commandPermissions: [...ALL_DEVICE_COMMANDS],
src/api/routes/devices.routes.ts:0165           tags: [device.chipModel.toLowerCase(), device.mode, device.isOnline ? 'online' : 'offline'],
src/api/routes/devices.routes.ts:0166           updatedAt: new Date().toISOString()
src/api/routes/devices.routes.ts:0167       };
src/api/routes/devices.routes.ts:0168   }
src/api/routes/devices.routes.ts:0169
src/api/routes/devices.routes.ts:0170   function normalizeCommandPermissions(input: unknown, fallback: string[]): string[] {
src/api/routes/devices.routes.ts:0171       if (!Array.isArray(input)) {
src/api/routes/devices.routes.ts:0172           return fallback;
src/api/routes/devices.routes.ts:0173       }
src/api/routes/devices.routes.ts:0174
src/api/routes/devices.routes.ts:0175       const next = input
src/api/routes/devices.routes.ts:0176           .map((value) => String(value || '').trim())
src/api/routes/devices.routes.ts:0177           .filter((value, index, list) => value && ALL_DEVICE_COMMANDS.includes(value) && list.indexOf(value) === index);
src/api/routes/devices.routes.ts:0178
src/api/routes/devices.routes.ts:0179       return next.length > 0 ? next : fallback;
src/api/routes/devices.routes.ts:0180   }
src/api/routes/devices.routes.ts:0181
src/api/routes/devices.routes.ts:0182   function serializeDevice(device: DeviceInfo) {
src/api/routes/devices.routes.ts:0183       return {
src/api/routes/devices.routes.ts:0184           ...device,
src/api/routes/devices.routes.ts:0185           lastHeartbeat: device.lastHeartbeat.toISOString(),
src/api/routes/devices.routes.ts:0186           physicalAuth: {
src/api/routes/devices.routes.ts:0187               hasPublicKey: Boolean(device.physicalAuth?.publicKeyPem),
src/api/routes/devices.routes.ts:0188               monotonicCounter: device.physicalAuth?.monotonicCounter || 0,
```

backend/src/api/routes/devices.routes.ts:189-238

```
src/api/routes/devices.routes.ts:0189              lastConfirmedAt: device.physicalAuth?.lastConfirmedAt || null
src/api/routes/devices.routes.ts:0190          }
src/api/routes/devices.routes.ts:0191      };
src/api/routes/devices.routes.ts:0192  }
src/api/routes/devices.routes.ts:0193
src/api/routes/devices.routes.ts:0194  function ensureDeviceConfig(device: DeviceInfo): DeviceConfig {
src/api/routes/devices.routes.ts:0195      const existing = deviceConfigs.get(device.deviceId);
src/api/routes/devices.routes.ts:0196      if (existing) {
src/api/routes/devices.routes.ts:0197          return existing;
src/api/routes/devices.routes.ts:0198      }
src/api/routes/devices.routes.ts:0199      const next = buildDefaultConfig(device);
src/api/routes/devices.routes.ts:0200      deviceConfigs.set(device.deviceId, next);
src/api/routes/devices.routes.ts:0201      return next;
src/api/routes/devices.routes.ts:0202  }
src/api/routes/devices.routes.ts:0203
src/api/routes/devices.routes.ts:0204  function getDeviceCommandHistory(deviceId: string, limit = 10): DeviceCommandRecord[] {
src/api/routes/devices.routes.ts:0205      return (deviceCommands.get(deviceId) || []).slice(0, limit);
src/api/routes/devices.routes.ts:0206  }
src/api/routes/devices.routes.ts:0207
src/api/routes/devices.routes.ts:0208  function pushDeviceCommand(record: DeviceCommandRecord): void {
src/api/routes/devices.routes.ts:0209      const rows = deviceCommands.get(record.deviceId) || [];
src/api/routes/devices.routes.ts:0210      rows.unshift(record);
src/api/routes/devices.routes.ts:0211      if (rows.length > 50) {
src/api/routes/devices.routes.ts:0212          rows.length = 50;
src/api/routes/devices.routes.ts:0213      }
src/api/routes/devices.routes.ts:0214      deviceCommands.set(record.deviceId, rows);
src/api/routes/devices.routes.ts:0215  }
src/api/routes/devices.routes.ts:0216
src/api/routes/devices.routes.ts:0217  function getLatestConfirmationSessionSafe(deviceId: string): LatestConfirmationSession {
src/api/routes/devices.routes.ts:0218      try {
src/api/routes/devices.routes.ts:0219          const module = require('../../services/device-control.service') as {
src/api/routes/devices.routes.ts:0220              deviceControlService?: {
src/api/routes/devices.routes.ts:0221                  getLatestConfirmationSession?: (value: string) => unknown;
src/api/routes/devices.routes.ts:0222              };
src/api/routes/devices.routes.ts:0223          };
src/api/routes/devices.routes.ts:0224          const session = module.deviceControlService?.getLatestConfirmationSession?.(deviceId);
src/api/routes/devices.routes.ts:0225          return asRecord(session);
src/api/routes/devices.routes.ts:0226      } catch {
src/api/routes/devices.routes.ts:0227          return null;
src/api/routes/devices.routes.ts:0228      }
src/api/routes/devices.routes.ts:0229  }
src/api/routes/devices.routes.ts:0230
src/api/routes/devices.routes.ts:0231  export function getDeviceById(deviceId: string): DeviceInfo | undefined {
src/api/routes/devices.routes.ts:0232      return devices.get(deviceId);
src/api/routes/devices.routes.ts:0233  }
src/api/routes/devices.routes.ts:0234
src/api/routes/devices.routes.ts:0235  export function patchDeviceRuntime(deviceId: string, patch: Partial<DeviceInfo>): void {
src/api/routes/devices.routes.ts:0236      const device = devices.get(deviceId);
src/api/routes/devices.routes.ts:0237      if (!device) {
src/api/routes/devices.routes.ts:0238          return;
```

backend/src/api/routes/devices.routes.ts:239-288

```
src/api/routes/devices.routes.ts:0239       }
src/api/routes/devices.routes.ts:0240
src/api/routes/devices.routes.ts:0241       const next: DeviceInfo = {
src/api/routes/devices.routes.ts:0242           ...device,
src/api/routes/devices.routes.ts:0243           ...patch,
src/api/routes/devices.routes.ts:0244           systemInfo: patch.systemInfo ? { ...(device.systemInfo || {}), ...patch.systemInfo } : device.systemInfo,
src/api/routes/devices.routes.ts:0245           physicalAuth: patch.physicalAuth ? { ...(device.physicalAuth || {}), ...patch.physicalAuth } : device.physicalAuth
src/api/routes/devices.routes.ts:0246       };
src/api/routes/devices.routes.ts:0247
src/api/routes/devices.routes.ts:0248       devices.set(deviceId, next);
src/api/routes/devices.routes.ts:0249   }
src/api/routes/devices.routes.ts:0250
src/api/routes/devices.routes.ts:0251   export function setDeviceCommandTransport(
src/api/routes/devices.routes.ts:0252       transport: ((deviceId: string, command: string, params?: DeviceCommandParams) => Promise<boolean> | boolean) | null
src/api/routes/devices.routes.ts:0253   ): void {
src/api/routes/devices.routes.ts:0254       deviceCommandTransport = transport;
src/api/routes/devices.routes.ts:0255   }
src/api/routes/devices.routes.ts:0256
src/api/routes/devices.routes.ts:0257   export async function executeDeviceCommandInternal(input: {
src/api/routes/devices.routes.ts:0258       deviceId: string;
src/api/routes/devices.routes.ts:0259       command: string;
src/api/routes/devices.routes.ts:0260       params?: DeviceCommandParams;
src/api/routes/devices.routes.ts:0261       actorId: string;
src/api/routes/devices.routes.ts:0262       actorEmail?: string;
src/api/routes/devices.routes.ts:0263       actorIp?: string;
src/api/routes/devices.routes.ts:0264       actionId?: string;
src/api/routes/devices.routes.ts:0265   }): Promise<{ success: boolean; record?: DeviceCommandRecord; message: string; reason?: string }> {
src/api/routes/devices.routes.ts:0266       const device = devices.get(input.deviceId);
src/api/routes/devices.routes.ts:0267       if (!device) {
src/api/routes/devices.routes.ts:0268           return { success: false, message: 'Device not found.', reason: 'device_not_found' };
src/api/routes/devices.routes.ts:0269       }
src/api/routes/devices.routes.ts:0270
src/api/routes/devices.routes.ts:0271       const config = ensureDeviceConfig(device);
src/api/routes/devices.routes.ts:0272       if (!config.commandPermissions.includes(input.command)) {
src/api/routes/devices.routes.ts:0273           return { success: false, message: `Command ${input.command} is not permitted for this device.`, reason: 'command_not_permitted' };
src/api/routes/devices.routes.ts:0274       }
src/api/routes/devices.routes.ts:0275
src/api/routes/devices.routes.ts:0276       if (!device.isOnline) {
src/api/routes/devices.routes.ts:0277           return { success: false, message: 'Device is offline.', reason: 'device_offline' };
src/api/routes/devices.routes.ts:0278       }
src/api/routes/devices.routes.ts:0279
src/api/routes/devices.routes.ts:0280       if (deviceCommandTransport) {
src/api/routes/devices.routes.ts:0281           const accepted = await Promise.resolve(deviceCommandTransport(input.deviceId, input.command, input.params || {}));
src/api/routes/devices.routes.ts:0282           if (!accepted) {
src/api/routes/devices.routes.ts:0283               return { success: false, message: `Device gateway rejected command ${input.command}.`, reason: 'device_gateway_rejected' };
src/api/routes/devices.routes.ts:0284           }
src/api/routes/devices.routes.ts:0285       }
src/api/routes/devices.routes.ts:0286
src/api/routes/devices.routes.ts:0287       const now = new Date().toISOString();
src/api/routes/devices.routes.ts:0288       const commandMessages: Record<string, string> = {
```

backend/src/api/routes/devices.routes.ts:289-338

```
src/api/routes/devices.routes.ts:0289           collect: `Telemetry collection completed on ${input.deviceId}`,
src/api/routes/devices.routes.ts:0290           'sync-config': `Configuration synchronized on ${input.deviceId}`,
src/api/routes/devices.routes.ts:0291           ota_update: `OTA workflow acknowledged on ${input.deviceId}`,
src/api/routes/devices.routes.ts:0292           host_backup: `Host backup requested on ${input.deviceId}`,
src/api/routes/devices.routes.ts:0293           host_install: `Host install workflow queued on ${input.deviceId}`,
src/api/routes/devices.routes.ts:0294           reboot: `Reboot command executed on ${input.deviceId}`,
src/api/routes/devices.routes.ts:0295           host_remove: `Host removal workflow queued on ${input.deviceId}`,
src/api/routes/devices.routes.ts:0296           recovery_flash: `Recovery flash workflow queued on ${input.deviceId}`,
src/api/routes/devices.routes.ts:0297           factory reset: `Factory reset workflow queued on ${input.deviceId}`,
src/api/routes/devices.routes.ts:0298           diagnostic unlock: `Diagnostic unlock workflow queued on ${input.deviceId}`,
src/api/routes/devices.routes.ts:0299           vehicle_ota_flash: `Vehicle OTA flash workflow queued on ${input.deviceId}`,
src/api/routes/devices.routes.ts:0300           cellular_breaker_assert: `Cellular breaker asserted on ${input.deviceId}`,
src/api/routes/devices.routes.ts:0301           cellular_breaker_release: `Cellular breaker released on ${input.deviceId}`
src/api/routes/devices.routes.ts:0302       };
src/api/routes/devices.routes.ts:0303       const record: DeviceCommandRecord = {
src/api/routes/devices.routes.ts:0304           id: input.actionId || `${Date.now().toString(36)}-${Math.random().toString(36).slice(2, 8)}`,
src/api/routes/devices.routes.ts:0305           deviceId: input.deviceId,
src/api/routes/devices.routes.ts:0306           command: input.command,
src/api/routes/devices.routes.ts:0307           status: 'completed',
src/api/routes/devices.routes.ts:0308           message: commandMessages[input.command] || `Command "${input.command}" executed successfully on ${input.deviceId}`,
src/api/routes/devices.routes.ts:0309           executedBy: input.actorEmail || input.actorId || 'unknown',
src/api/routes/devices.routes.ts:0310           createdAt: now,
src/api/routes/devices.routes.ts:0311           completedAt: now,
src/api/routes/devices.routes.ts:0312           params: input.params || {}
src/api/routes/devices.routes.ts:0313       };
src/api/routes/devices.routes.ts:0314
src/api/routes/devices.routes.ts:0315       pushDeviceCommand(record);
src/api/routes/devices.routes.ts:0316       addDeviceEvent(input.deviceId, 'command', {
src/api/routes/devices.routes.ts:0317           action: input.command,
src/api/routes/devices.routes.ts:0318           params: input.params || {},
src/api/routes/devices.routes.ts:0319           status: record.status,
src/api/routes/devices.routes.ts:0320           message: record.message,
src/api/routes/devices.routes.ts:0321           queuedBy: record.executedBy,
src/api/routes/devices.routes.ts:0322           actionId: input.actionId || null
src/api/routes/devices.routes.ts:0323       });
src/api/routes/devices.routes.ts:0324
src/api/routes/devices.routes.ts:0325       if (input.command === 'collect') {
src/api/routes/devices.routes.ts:0326           device.lastHeartbeat = new Date();
src/api/routes/devices.routes.ts:0327           device.freeHeap = Math.max(64_000, device.freeHeap - 2_048);
src/api/routes/devices.routes.ts:0328       }
src/api/routes/devices.routes.ts:0329
src/api/routes/devices.routes.ts:0330       if (input.command === 'reboot') {
src/api/routes/devices.routes.ts:0331           device.lastHeartbeat = new Date();
src/api/routes/devices.routes.ts:0332           device.status = device.status
src/api/routes/devices.routes.ts:0333               ? { ...device.status, uptime: 0, cpuUsage: Math.max(5, Math.round(device.status.cpuUsage * 0.6)) }
src/api/routes/devices.routes.ts:0334               : device.status;
src/api/routes/devices.routes.ts:0335           device.pendingPhysicalApproval = false;
src/api/routes/devices.routes.ts:0336           device.hostActionState = 'completed';
src/api/routes/devices.routes.ts:0337       }
src/api/routes/devices.routes.ts:0338
```

backend/src/api/routes/devices.routes.ts:339-388

```
src/api/routes/devices.routes.ts:0339       if (input.command === 'host_backup') {
src/api/routes/devices.routes.ts:0340           device.hostActionState = 'running';
src/api/routes/devices.routes.ts:0341       }
src/api/routes/devices.routes.ts:0342
src/api/routes/devices.routes.ts:0343       if (['host_install', 'host_remove', 'recovery_flash', 'factory_reset'].includes(input.command)) {
src/api/routes/devices.routes.ts:0344           device.pendingPhysicalApproval = false;
src/api/routes/devices.routes.ts:0345           device.hostActionState = 'running';
src/api/routes/devices.routes.ts:0346       }
src/api/routes/devices.routes.ts:0347
src/api/routes/devices.routes.ts:0348       if (['diagnostic unlock', 'vehicle ota flash'].includes(input.command)) {
src/api/routes/devices.routes.ts:0349           device.pendingPhysicalApproval = false;
src/api/routes/devices.routes.ts:0350           device.hostActionState = 'running';
src/api/routes/devices.routes.ts:0351       }
src/api/routes/devices.routes.ts:0352
src/api/routes/devices.routes.ts:0353       if (input.command === 'cellular_breaker_assert') {
src/api/routes/devices.routes.ts:0354           device.pendingPhysicalApproval = false;
src/api/routes/devices.routes.ts:0355           device.hostActionState = 'completed';
src/api/routes/devices.routes.ts:0356           device.interlockState = 'asserted';
src/api/routes/devices.routes.ts:0357       }
src/api/routes/devices.routes.ts:0358
src/api/routes/devices.routes.ts:0359       if (input.command === 'cellular breaker release') {
src/api/routes/devices.routes.ts:0360           device.pendingPhysicalApproval = false;
src/api/routes/devices.routes.ts:0361           device.hostActionState = 'completed';
src/api/routes/devices.routes.ts:0362           device.interlockState = device.externalPowerPresent ? 'armed' : 'disabled';
src/api/routes/devices.routes.ts:0363       }
src/api/routes/devices.routes.ts:0364
src/api/routes/devices.routes.ts:0365       devices.set(input.deviceId, device);
src/api/routes/devices.routes.ts:0366       try {
src/api/routes/devices.routes.ts:0367           const asset = ensureDeviceAsset(device);
src/api/routes/devices.routes.ts:0368           syncDeviceSignals(device, asset.assetId);
src/api/routes/devices.routes.ts:0369       } catch {
src/api/routes/devices.routes.ts:0370           // Ignore when operations database is unavailable.
src/api/routes/devices.routes.ts:0371       }
src/api/routes/devices.routes.ts:0372
src/api/routes/devices.routes.ts:0373       logger.info('Command sent to device', {
src/api/routes/devices.routes.ts:0374           deviceId: input.deviceId,
src/api/routes/devices.routes.ts:0375           command: input.command,
src/api/routes/devices.routes.ts:0376           params: input.params || {},
src/api/routes/devices.routes.ts:0377           actionId: input.actionId || null
src/api/routes/devices.routes.ts:0378       });
src/api/routes/devices.routes.ts:0379
src/api/routes/devices.routes.ts:0380       return {
src/api/routes/devices.routes.ts:0381           success: true,
src/api/routes/devices.routes.ts:0382           record,
src/api/routes/devices.routes.ts:0383           message: record.message
src/api/routes/devices.routes.ts:0384       };
src/api/routes/devices.routes.ts:0385   }
src/api/routes/devices.routes.ts:0386
src/api/routes/devices.routes.ts:0387   commandDispatcherService.setDeviceAdapter({
src/api/routes/devices.routes.ts:0388       getDevice: getDeviceById,
```

backend/src/api/routes/devices.routes.ts:389-438

```
src/api/routes/devices.routes.ts:0389       executeCommand: executeDeviceCommandInternal
src/api/routes/devices.routes.ts:0390   });
src/api/routes/devices.routes.ts:0391
src/api/routes/devices.routes.ts:0392   function ensureDeviceAsset(device: DeviceInfo) {
src/api/routes/devices.routes.ts:0393       const config = ensureDeviceConfig(device);
src/api/routes/devices.routes.ts:0394       return securityOperationsService.upsertAsset({
src/api/routes/devices.routes.ts:0395           tenantId: PLATFORM_TENANT_ID,
src/api/routes/devices.routes.ts:0396           assetType: 'device',
src/api/routes/devices.routes.ts:0397           externalRef: device.deviceId,
src/api/routes/devices.routes.ts:0398           displayName: config.displayName || device.deviceId,
src/api/routes/devices.routes.ts:0399           primaryAddress: device.systemInfo?.hostname,
src/api/routes/devices.routes.ts:0400           status: device.isOnline ? 'online' : 'offline',
src/api/routes/devices.routes.ts:0401           metadata: {
src/api/routes/devices.routes.ts:0402               chipModel: device.chipModel,
src/api/routes/devices.routes.ts:0403               firmwareVersion: device.version,
src/api/routes/devices.routes.ts:0404               group: config.group,
src/api/routes/devices.routes.ts:0405               site: config.site,
src/api/routes/devices.routes.ts:0406               networkProfile: config.networkProfile,
src/api/routes/devices.routes.ts:0407               scanProfile: config.scanProfile,
src/api/routes/devices.routes.ts:0408               tags: config.tags
src/api/routes/devices.routes.ts:0409           }
src/api/routes/devices.routes.ts:0410       });
src/api/routes/devices.routes.ts:0411   }
src/api/routes/devices.routes.ts:0412
src/api/routes/devices.routes.ts:0413   function syncDeviceSignals(device: DeviceInfo, assetId?: string): void {
src/api/routes/devices.routes.ts:0414       const target = device.systemInfo?.hostname || device.deviceId;
src/api/routes/devices.routes.ts:0415       const heartbeatAgeMs = Date.now() - device.lastHeartbeat.getTime();
src/api/routes/devices.routes.ts:0416       const staleHeartbeat = device.isOnline && heartbeatAgeMs > 15 * 60 * 1000;
src/api/routes/devices.routes.ts:0417
src/api/routes/devices.routes.ts:0418       securityOperationsService.syncSignalFinding({
src/api/routes/devices.routes.ts:0419           tenantId: PLATFORM TENANT ID,
src/api/routes/devices.routes.ts:0420           active: !device.isOnline || staleHeartbeat,
src/api/routes/devices.routes.ts:0421           assetId,
src/api/routes/devices.routes.ts:0422           sourceType: 'device-health',
src/api/routes/devices.routes.ts:0423           sourceRef: device.deviceId,
src/api/routes/devices.routes.ts:0424           target,
src/api/routes/devices.routes.ts:0425           category: 'device-health',
src/api/routes/devices.routes.ts:0426           severity: 'high',
src/api/routes/devices.routes.ts:0427           title: !device.isOnline ? 'Device Offline' : 'Device Heartbeat Stale',
src/api/routes/devices.routes.ts:0428           description: !device.isOnline
src/api/routes/devices.routes.ts:0429               ? `Device ${device.deviceId} is offline and not reporting telemetry.`
src/api/routes/devices.routes.ts:0430               : `Device ${device.deviceId} has not reported for more than 15 minutes.`,
src/api/routes/devices.routes.ts:0431           evidence: {
src/api/routes/devices.routes.ts:0432               deviceId: device.deviceId,
src/api/routes/devices.routes.ts:0433               lastHeartbeat: device.lastHeartbeat.toISOString(),
src/api/routes/devices.routes.ts:0434               isOnline: device.isOnline,
src/api/routes/devices.routes.ts:0435               staleHeartbeat
src/api/routes/devices.routes.ts:0436           },
src/api/routes/devices.routes.ts:0437           remediation: {
src/api/routes/devices.routes.ts:0438               title: 'Recover device connectivity',
```

backend/src/api/routes/devices.routes.ts:439-488

```
src/api/routes/devices.routes.ts:0439                owner: 'operations',
src/api/routes/devices.routes.ts:0440                priority: 'high',
src/api/routes/devices.routes.ts:0441                suggestedSteps: [
src/api/routes/devices.routes.ts:0442                    'Check the device power and network connectivity.',
src/api/routes/devices.routes.ts:0443                    'Verify the gateway can still reach the device.',
src/api/routes/devices.routes.ts:0444                    'Reboot or re-enroll the device if telemetry does not resume.'
src/api/routes/devices.routes.ts:0445                ]
src/api/routes/devices.routes.ts:0446            },
src/api/routes/devices.routes.ts:0447            fingerprint: `device:${device.deviceId}:health:availability`,
src/api/routes/devices.routes.ts:0448            createAlert: true,
src/api/routes/devices.routes.ts:0449            alertType: 'system'
src/api/routes/devices.routes.ts:0450        });
src/api/routes/devices.routes.ts:0451
src/api/routes/devices.routes.ts:0452        securityOperationsService.syncSignalFinding({
src/api/routes/devices.routes.ts:0453            tenantId: PLATFORM_TENANT_ID,
src/api/routes/devices.routes.ts:0454            active: Boolean(device.status && device.status.cpuUsage >= 90),
src/api/routes/devices.routes.ts:0455            assetId,
src/api/routes/devices.routes.ts:0456            sourceType: 'device-health',
src/api/routes/devices.routes.ts:0457            sourceRef: device.deviceId,
src/api/routes/devices.routes.ts:0458            target,
src/api/routes/devices.routes.ts:0459            category: 'device-health',
src/api/routes/devices.routes.ts:0460            severity: 'high',
src/api/routes/devices.routes.ts:0461            title: 'Device CPU Saturation',
src/api/routes/devices.routes.ts:0462            description: `Device ${device.deviceId} CPU usage is ${device.status?.cpuUsage || 0}%.`,
src/api/routes/devices.routes.ts:0463            evidence: {
src/api/routes/devices.routes.ts:0464                deviceId: device.deviceId,
src/api/routes/devices.routes.ts:0465                cpuUsage: device.status?.cpuUsage || 0
src/api/routes/devices.routes.ts:0466            },
src/api/routes/devices.routes.ts:0467            remediation: {
src/api/routes/devices.routes.ts:0468                title: 'Reduce device CPU pressure',
src/api/routes/devices.routes.ts:0469                owner: 'operations',
src/api/routes/devices.routes.ts:0470                priority: 'high',
src/api/routes/devices.routes.ts:0471                suggestedSteps: [
src/api/routes/devices.routes.ts:0472                    'Inspect the running workload and recent commands.',
src/api/routes/devices.routes.ts:0473                    'Reduce concurrent collection tasks or reboot the device.',
src/api/routes/devices.routes.ts:0474                    'Validate CPU returns to a stable baseline after mitigation.'
src/api/routes/devices.routes.ts:0475                ]
src/api/routes/devices.routes.ts:0476            },
src/api/routes/devices.routes.ts:0477            fingerprint: `device:${device.deviceId}:health:cpu`,
src/api/routes/devices.routes.ts:0478            createAlert: true,
src/api/routes/devices.routes.ts:0479            alertType: 'system'
src/api/routes/devices.routes.ts:0480        });
src/api/routes/devices.routes.ts:0481
src/api/routes/devices.routes.ts:0482        securityOperationsService.syncSignalFinding({
src/api/routes/devices.routes.ts:0483            tenantId: PLATFORM_TENANT_ID,
src/api/routes/devices.routes.ts:0484            active: Boolean(device.status && device.status.memoryUsage >= 85),
src/api/routes/devices.routes.ts:0485            assetId,
src/api/routes/devices.routes.ts:0486            sourceType: 'device-health',
src/api/routes/devices.routes.ts:0487            sourceRef: device.deviceId,
src/api/routes/devices.routes.ts:0488            target,
```

backend/src/api/routes/devices.routes.ts:489-538

```
src/api/routes/devices.routes.ts:0489           category: 'device-health',
src/api/routes/devices.routes.ts:0490           severity: 'medium',
src/api/routes/devices.routes.ts:0491           title: 'Device Memory Pressure',
src/api/routes/devices.routes.ts:0492           description: `Device ${device.deviceId} memory usage is ${device.status?.memoryUsage || 0}%.`,
src/api/routes/devices.routes.ts:0493           evidence: {
src/api/routes/devices.routes.ts:0494               deviceId: device.deviceId,
src/api/routes/devices.routes.ts:0495               memoryUsage: device.status?.memoryUsage || 0,
src/api/routes/devices.routes.ts:0496               freeHeap: device.freeHeap
src/api/routes/devices.routes.ts:0497           },
src/api/routes/devices.routes.ts:0498           remediation: {
src/api/routes/devices.routes.ts:0499               title: 'Stabilize device memory usage',
src/api/routes/devices.routes.ts:0500               owner: 'operations',
src/api/routes/devices.routes.ts:0501               priority: 'medium',
src/api/routes/devices.routes.ts:0502               suggestedSteps: [
src/api/routes/devices.routes.ts:0503                   'Collect a fresh heap snapshot from the device.',
src/api/routes/devices.routes.ts:0504                   'Reduce the active workload or restart memory-heavy modules.',
src/api/routes/devices.routes.ts:0505                   'Confirm free heap recovers after the change.'
src/api/routes/devices.routes.ts:0506               ]
src/api/routes/devices.routes.ts:0507           },
src/api/routes/devices.routes.ts:0508           fingerprint: `device:${device.deviceId}:health:memory`,
src/api/routes/devices.routes.ts:0509           createAlert: false,
src/api/routes/devices.routes.ts:0510           alertType: 'system'
src/api/routes/devices.routes.ts:0511       });
src/api/routes/devices.routes.ts:0512
src/api/routes/devices.routes.ts:0513       securityOperationsService.syncSignalFinding({
src/api/routes/devices.routes.ts:0514           tenantId: PLATFORM_TENANT_ID,
src/api/routes/devices.routes.ts:0515           active: Boolean(device.status && device.status.wifiRssi <= -80),
src/api/routes/devices.routes.ts:0516           assetId,
src/api/routes/devices.routes.ts:0517           sourceType: 'device-health',
src/api/routes/devices.routes.ts:0518           sourceRef: device.deviceId,
src/api/routes/devices.routes.ts:0519           target,
src/api/routes/devices.routes.ts:0520           category: 'device-connectivity',
src/api/routes/devices.routes.ts:0521           severity: 'medium',
src/api/routes/devices.routes.ts:0522           title: 'Device Signal Weak',
src/api/routes/devices.routes.ts:0523           description: `Device ${device.deviceId} signal strength dropped to ${device.status?.wifiRssi || 0} dBm.`,
src/api/routes/devices.routes.ts:0524           evidence: {
src/api/routes/devices.routes.ts:0525               deviceId: device.deviceId,
src/api/routes/devices.routes.ts:0526               wifiRssi: device.status?.wifiRssi || 0
src/api/routes/devices.routes.ts:0527           },
src/api/routes/devices.routes.ts:0528           remediation: {
src/api/routes/devices.routes.ts:0529               title: 'Improve device link quality',
src/api/routes/devices.routes.ts:0530               owner: 'operations',
src/api/routes/devices.routes.ts:0531               priority: 'medium',
src/api/routes/devices.routes.ts:0532               suggestedSteps: [
src/api/routes/devices.routes.ts:0533                   'Validate access point proximity and interference.',
src/api/routes/devices.routes.ts:0534                   'Move the device or reassign the wireless profile if needed.',
src/api/routes/devices.routes.ts:0535                   'Confirm telemetry stability after the signal recovers.'
src/api/routes/devices.routes.ts:0536               ]
src/api/routes/devices.routes.ts:0537           },
src/api/routes/devices.routes.ts:0538           fingerprint: `device:${device.deviceId}:health:wifi`,
```

backend/src/api/routes/devices.routes.ts:539-588

```
src/api/routes/devices.routes.ts:0539          createAlert: false,
src/api/routes/devices.routes.ts:0540          alertType: 'system'
src/api/routes/devices.routes.ts:0541      });
src/api/routes/devices.routes.ts:0542  }
src/api/routes/devices.routes.ts:0543
src/api/routes/devices.routes.ts:0544  function recordDeviceAction(device: DeviceInfo, input: { title: string; status: 'pending' | 'running' | 'completed' | 'failed'; operator: string; rol
src/api/routes/devices.routes.ts:0545      try {
src/api/routes/devices.routes.ts:0546          const asset = ensureDeviceAsset(device);
src/api/routes/devices.routes.ts:0547          securityOperationsService.recordActionRun({
src/api/routes/devices.routes.ts:0548              tenantId: PLATFORM TENANT ID,
src/api/routes/devices.routes.ts:0549              assetId: asset.assetId,
src/api/routes/devices.routes.ts:0550              title: input.title,
src/api/routes/devices.routes.ts:0551              status: input.status,
src/api/routes/devices.routes.ts:0552              operator: input.operator,
src/api/routes/devices.routes.ts:0553              rollbackNotes: input.rollbackNotes,
src/api/routes/devices.routes.ts:0554              runLog: input.runLog
src/api/routes/devices.routes.ts:0555          });
src/api/routes/devices.routes.ts:0556      } catch {
src/api/routes/devices.routes.ts:0557          // Ignore when operations database is unavailable in lightweight tests.
src/api/routes/devices.routes.ts:0558      }
src/api/routes/devices.routes.ts:0559  }
src/api/routes/devices.routes.ts:0560
src/api/routes/devices.routes.ts:0561  function summarizeSecurity(device: DeviceInfo) {
src/api/routes/devices.routes.ts:0562      const commandHistory = getDeviceCommandHistory(device.deviceId, 5);
src/api/routes/devices.routes.ts:0563      try {
src/api/routes/devices.routes.ts:0564          const asset = ensureDeviceAsset(device);
src/api/routes/devices.routes.ts:0565          if (device.systemInfo?.hostname) {
src/api/routes/devices.routes.ts:0566              securityOperationsService.upsertAsset({
src/api/routes/devices.routes.ts:0567                  tenantId: PLATFORM_TENANT_ID,
src/api/routes/devices.routes.ts:0568                  assetType: 'host',
src/api/routes/devices.routes.ts:0569                  externalRef: device.systemInfo.hostname,
src/api/routes/devices.routes.ts:0570                  displayName: device.systemInfo.hostname,
src/api/routes/devices.routes.ts:0571                  primaryAddress: device.systemInfo.hostname,
src/api/routes/devices.routes.ts:0572                  status: device.isOnline ? 'online' : 'offline',
src/api/routes/devices.routes.ts:0573                  metadata: {
src/api/routes/devices.routes.ts:0574                      deviceId: device.deviceId,
src/api/routes/devices.routes.ts:0575                      chipModel: device.chipModel
src/api/routes/devices.routes.ts:0576                  }
src/api/routes/devices.routes.ts:0577              });
src/api/routes/devices.routes.ts:0578          }
src/api/routes/devices.routes.ts:0579          syncDeviceSignals(device, asset.assetId);
src/api/routes/devices.routes.ts:0580          const context = securityOperationsService.getOperationalContextForExternalRefs(
src/api/routes/devices.routes.ts:0581              PLATFORM TENANT ID,
src/api/routes/devices.routes.ts:0582              [device.deviceId, device.systemInfo?.hostname || ''],
src/api/routes/devices.routes.ts:0583              ['device', 'host']
src/api/routes/devices.routes.ts:0584          );
src/api/routes/devices.routes.ts:0585          return {
src/api/routes/devices.routes.ts:0586              asset: context.assets.find((item) => item.assetId === asset.assetId) || context.assets[0] || asset,
src/api/routes/devices.routes.ts:0587              findings: context.findings,
src/api/routes/devices.routes.ts:0588              alerts: context.alerts,
```

backend/src/api/routes/devices.routes.ts:589-638

```
src/api/routes/devices.routes.ts:0589              actions: context.actions,
src/api/routes/devices.routes.ts:0590              recentScans: context.scanRuns,
src/api/routes/devices.routes.ts:0591              serviceExposure: context.serviceExposure,
src/api/routes/devices.routes.ts:0592              relatedSessions: context.trafficSessions,
src/api/routes/devices.routes.ts:0593              latestCommand: commandHistory[0] || null
src/api/routes/devices.routes.ts:0594          };
src/api/routes/devices.routes.ts:0595      } catch {
src/api/routes/devices.routes.ts:0596          return {
src/api/routes/devices.routes.ts:0597              asset: null,
src/api/routes/devices.routes.ts:0598              findings: [],
src/api/routes/devices.routes.ts:0599              alerts: [],
src/api/routes/devices.routes.ts:0600              actions: [],
src/api/routes/devices.routes.ts:0601              recentScans: [],
src/api/routes/devices.routes.ts:0602              serviceExposure: [],
src/api/routes/devices.routes.ts:0603              relatedSessions: [],
src/api/routes/devices.routes.ts:0604              latestCommand: commandHistory[0] || null
src/api/routes/devices.routes.ts:0605          };
src/api/routes/devices.routes.ts:0606      }
src/api/routes/devices.routes.ts:0607  }
src/api/routes/devices.routes.ts:0608
src/api/routes/devices.routes.ts:0609  /**
src/api/routes/devices.routes.ts:0610   * GET /api/devices
src/api/routes/devices.routes.ts:0611   * Get all ESP32 device list
src/api/routes/devices.routes.ts:0612   */
src/api/routes/devices.routes.ts:0613  router.get('/', (_req: Request, res: Response) => {
src/api/routes/devices.routes.ts:0614      try {
src/api/routes/devices.routes.ts:0615          const deviceList = Array.from(devices.values()).map(device => ({
src/api/routes/devices.routes.ts:0616              ...(() => {
src/api/routes/devices.routes.ts:0617                  const config = ensureDeviceConfig(device);
src/api/routes/devices.routes.ts:0618                  try {
src/api/routes/devices.routes.ts:0619                      securityOperationsService.upsertAsset({
src/api/routes/devices.routes.ts:0620                          tenantId: PLATFORM_TENANT_ID,
src/api/routes/devices.routes.ts:0621                          assetType: 'device',
src/api/routes/devices.routes.ts:0622                          externalRef: device.deviceId,
src/api/routes/devices.routes.ts:0623                          displayName: device.deviceId,
src/api/routes/devices.routes.ts:0624                          primaryAddress: device.systemInfo?.hostname,
src/api/routes/devices.routes.ts:0625                          status: device.isOnline ? 'online' : 'offline',
src/api/routes/devices.routes.ts:0626                          metadata: {
src/api/routes/devices.routes.ts:0627                              chipModel: device.chipModel,
src/api/routes/devices.routes.ts:0628                              firmwareVersion: device.version,
src/api/routes/devices.routes.ts:0629                              group: config.group,
src/api/routes/devices.routes.ts:0630                              site: config.site
src/api/routes/devices.routes.ts:0631                          }
src/api/routes/devices.routes.ts:0632                      });
src/api/routes/devices.routes.ts:0633                  } catch {
src/api/routes/devices.routes.ts:0634                      // Ignore when operations database is not initialized.
src/api/routes/devices.routes.ts:0635                  }
src/api/routes/devices.routes.ts:0636                  return {};
src/api/routes/devices.routes.ts:0637              })(),
src/api/routes/devices.routes.ts:0638              ...serializeDevice(device),
```

backend/src/api/routes/devices.routes.ts:639-688

```
src/api/routes/devices.routes.ts:0639              config: ensureDeviceConfig(device),
src/api/routes/devices.routes.ts:0640              securitySummary: summarizeSecurity(device),
src/api/routes/devices.routes.ts:0641              latestConfirmationSession: getLatestConfirmationSessionSafe(device.deviceId)
src/api/routes/devices.routes.ts:0642          }));
src/api/routes/devices.routes.ts:0643
src/api/routes/devices.routes.ts:0644          res.json({
src/api/routes/devices.routes.ts:0645              success: true,
src/api/routes/devices.routes.ts:0646              total: deviceList.length,
src/api/routes/devices.routes.ts:0647              online: deviceList.filter(d => d.isOnline).length,
src/api/routes/devices.routes.ts:0648              devices: deviceList,
src/api/routes/devices.routes.ts:0649          });
src/api/routes/devices.routes.ts:0650      } catch (error) {
src/api/routes/devices.routes.ts:0651          logger.error('Failed to get devices', { error: getErrorMessage(error) });
src/api/routes/devices.routes.ts:0652          res.status(500).json({ success: false, error: 'backend.internal_server_error' });
src/api/routes/devices.routes.ts:0653      }
src/api/routes/devices.routes.ts:0654  });
src/api/routes/devices.routes.ts:0655
src/api/routes/devices.routes.ts:0656  /**
src/api/routes/devices.routes.ts:0657   * GET /api/devices/codebook
src/api/routes/devices.routes.ts:0658   * NOTE: This static route must be defined BEFORE /:id routes
src/api/routes/devices.routes.ts:0659   */
src/api/routes/devices.routes.ts:0660  router.get('/codebook', (_req: Request, res: Response) => {
src/api/routes/devices.routes.ts:0661      res.json({
src/api/routes/devices.routes.ts:0662          success: true,
src/api/routes/devices.routes.ts:0663          codebook,
src/api/routes/devices.routes.ts:0664      });
src/api/routes/devices.routes.ts:0665  });
src/api/routes/devices.routes.ts:0666
src/api/routes/devices.routes.ts:0667  /**
src/api/routes/devices.routes.ts:0668   * POST /api/devices/codebook
src/api/routes/devices.routes.ts:0669   * NOTE: This static route must be defined BEFORE /:id routes
src/api/routes/devices.routes.ts:0670   */
src/api/routes/devices.routes.ts:0671  router.post('/codebook', (req: Request, res: Response) => {
src/api/routes/devices.routes.ts:0672      try {
src/api/routes/devices.routes.ts:0673          const body = asRecord(req.body);
src/api/routes/devices.routes.ts:0674          const entries = asRecord(body?.entries);
src/api/routes/devices.routes.ts:0675
src/api/routes/devices.routes.ts:0676          if (!entries) {
src/api/routes/devices.routes.ts:0677              return res.status(400).json({ success: false, error: 'backend.invalid_entries' });
src/api/routes/devices.routes.ts:0678          }
src/api/routes/devices.routes.ts:0679
src/api/routes/devices.routes.ts:0680          const maxEntries = 1000;
src/api/routes/devices.routes.ts:0681          const currentEntries = Object.keys(codebook).length;
src/api/routes/devices.routes.ts:0682          const newEntriesCount = Object.keys(entries).length;
src/api/routes/devices.routes.ts:0683
src/api/routes/devices.routes.ts:0684          if (currentEntries + newEntriesCount > maxEntries) {
src/api/routes/devices.routes.ts:0685              return res.status(400).json({
src/api/routes/devices.routes.ts:0686                  success: false,
src/api/routes/devices.routes.ts:0687                  error: `Too many entries. Maximum ${maxEntries} entries allowed.`
src/api/routes/devices.routes.ts:0688              });
```